ROTEIRO DE LABORATÓRIO · PROGRAMAÇÃO BACK-END

Rastreando ameaças: auditoria e logs para APIs seguras

2ª série | 4º bimestre | Semana 25 | Aula 3 | Servidores Web, API e Segurança das Aplicações

Atividade elaborada pelo Prof. Marcos Souza, baseado nos materiais didáticos fornecidos pela Sec. Educ. do Estado de São Paulo para o programa de Educação Profissional Paulista, curso Técnico em Desenvolvimento de Sistemas, na disciplina de Programação Back-End – 2ª série, 4º bimestre, Semana 25, Aula 3, tema "Rastreando ameaças: auditoria e logs para APIs seguras" (unidade Servidores Web, API e Segurança das Aplicações) — Laboratório, seção 2.

A ferramenta de inteligência artificial Claude, desenvolvida pela Anthropic, foi utilizada para otimizar esta atividade.

Modelo: Claude Opus 5.5 (claude-opus-5-5) · gerado em 02/11/2026, às 19:12 (horário de Brasília)

Integrantes do grupo

Nomes: _____

_____

Turma: _____

Contexto do laboratório

Continuidade	Segunda seção do laboratório: a API financeira com OAuth construída na aula 1 recebe auditoria e logs. Quem não fez a aula 1 usa o script desta aula, que recria a base completa.
Sistema operacional	Windows 10/11 — PowerShell 5.1+ como shell padrão
Ferramentas	Node.js 20 LTS+ (npm 10+), Git 2.40+, Visual Studio Code 1.85+, navegador com DevTools

Dependências npm	Nenhuma — só módulos nativos do Node (fs, crypto, http). ELK/Splunk aparecem como configuração de referência opcional.
Grupos	2 a 4 estudantes por grupo
Duração	2 aulas de 50 min (100 min): 10 estudo · 15 instrumentação · 45 execução (logs ao vivo, ataques, análise, integridade, rotação) · 20 plano e perguntas · 10 entrega
Objetivo	Registrar eventos críticos da API em formato estruturado e protegido, gerar alertas e bloqueio automático contra força bruta, analisar os logs com indicadores e redigir o plano de auditoria do grupo.

Cenário

> “Equipe, identificamos algumas anomalias nos logs da nossa API e precisamos revisar nosso processo de auditoria. Como vocês recomendariam ajustar nossa configuração e análise de logs para detectar e responder a incidentes de segurança?”
>
> Situação fictícia produzida pela SEDUC-SP

Arquitetura do laboratório

Componente	Arquivo	Porta	Responsabilidade
Módulo de auditoria	`src/auditoria.js`	—	JSON Lines, mascaramento, cifra AES-GCM, HMAC encadeado, rotação/retenção, alertas e bloqueio
Servidor de Autorização	`src/auth-server.js`	4000	Registra logins, tokens, reuso de refresh; responde 429 a IP bloqueado
API Financeira	`src/api-financeira.js`	5000	Registra acessos permitidos/negados, operações sensíveis e tokens adulterados
App Finanças	`src/cliente-web.js`	3000	Propaga o X-Request-Id para provedor e API (rastreio ponta a ponta)
Simulador	`src/simular-trafego.js`	—	Tráfego normal + 5 tipos de ataque com IPs simulados
Analisador	`src/analisar-logs.js`	—	Integridade, KPIs, correlação por IP/req_id e relatório HTML

Ciclo de auditoria implementado

1. **Gerar:** cada serviço grava eventos estruturados (quem, o quê, quando, de onde, resultado) com `req_id` e `jti` para correlação.
2. **Proteger:** segredos mascarados, dado sensível cifrado (AES-256-GCM) e cada linha assinada com HMAC encadeado.
3. **Armazenar:** rotação por tamanho e retenção por quantidade/idade; `logs/` fora do Git.
4. **Detectar:** regras em tempo real — força bruta (alerta + bloqueio 429), token adulterado e reuso de refresh token.
5. **Analisar e responder:** verificação de integridade, indicadores, IPs suspeitos, rastreio por `req_id` e recomendações.

Checklist de recursos

- [x] **Computador com Windows 10/11 e PowerShell 5.1+**
  Confira com `$PSVersionTable.PSVersion`
- [x] **Node.js 20 LTS ou superior (inclui npm 10+)**
  nodejs.org ou `winget install OpenJS.NodeJS.LTS`
- [x] **Git 2.40 ou superior**
  git-scm.com ou `winget install Git.Git`
- [x] **Visual Studio Code 1.85 ou superior**
  code.visualstudio.com ou `winget install Microsoft.VisualStudioCode`
- [x] **Projeto da aula 1 (`Documentos\atividade-oauth-api`)**
  Se não tiver, o script desta aula recria a base completa
- [x] **Extensão Log File Highlighter (emilast.LogFileHighlighter) — opcional**
  Colore níveis e datas ao abrir `logs\auditoria.log` no VS Code
- [x] **Navegador atualizado com DevTools (F12)**
  Para provocar o bloqueio por força bruta e ver o cabeçalho `X-Request-Id`
- [x] **Portas 3000, 4000 e 5000 livres**
  Encerre servidores antigos antes do `npm start`
- [x] **Arquivo de partida: `atividade-auditoria-logs-api_script.ps1`**
  Preparação e execução automática desta seção
- [x] **Material de referência SEDUC-SP (roteiro SISANO2C2B4S25A3AP)**
  Ferramentas de auditoria, configuração de logs e análise de dados

Modelo do plano: `docs/plano-auditoria.md`

Documento que o grupo completa na etapa de Execução

Procedimento passo a passo

Passo 1 Conferir ferramentas e o projeto da aula 1

PowerShell

Confira as versões e abra a pasta do projeto da aula 1. O `git log` deve mostrar os commits da aula 1.

PowerShell — comandos

```
node -v; npm.cmd -v; git --version
cd "$HOME\Documents\atividade-oauth-api"
git status
git log --oneline
```

Resultado esperado: Versões mínimas atendidas e histórico da aula 1 visível (ou decisão de usar o script).

✔ EXEMPLO DE SUCESSO

```
v22.11.0
10.9.0
git version 2.47.1.windows.1
3f9c2ab feat: laboratorio OAuth ...
```

✘ EXEMPLO DE FALHA TÍPICA

```
cd : Não é possível localizar o caminho '...\atividade-oauth-api' porque ele não existe.
```

Como corrigir: Você não tem a base da aula 1: rode o script desta aula (passo 2), que recria o projeto completo.

Passo 2 (Opcional) Preparação automática com o script

PowerShell

O script cria a branch `lab2-auditoria` (salvando antes o trabalho pendente), grava os arquivos desta seção **preservando os planos já escritos**, roda os 52 testes, sobe os servidores, confere o log de acesso, executa a simulação de ataques, valida alertas, integridade e ausência de segredos nos logs, e gera o relatório. Ao final mostra a tabela passo/ambiente/comando/status, lista o que é **manual** e salva `relatorio-`

`preparacao.csv`. Não faz login, push nem configura serviços externos. Quem usar o script confere os passos 3 a 10 e segue no passo 11.

PowerShell — comandos

```
cd $HOME\Downloads
powershell -ExecutionPolicy Bypass -File .\atividade-auditoria-logs-api_script.ps1
# opções: -Destino "C:\outra\pasta"   -SemServidor
```

Resultado esperado: Relatório final sem itens FALHOU.

✔ EXEMPLO DE SUCESSO

```
[OK    ] 11 Testes automáticos - OAuth 25 OK; Auditoria 27 OK
...
Sucesso: 20   Falhas: 0   ...   Manuais: 8
```

✘ EXEMPLO DE FALHA TÍPICA

```
[FALHOU] 06 Branch lab2-auditoria - error: Your local changes would be
overwritten by checkout
```

Como corrigir: Rode de novo: o script commita o trabalho pendente antes de trocar de branch. Persistindo, rode git status e resolva conflitos.

Passo 3 Criar a branch da aula 3 e conferir a base

Git

Guarde o estado da aula 1 e trabalhe numa branch nova. Os arquivos abaixo **não mudam** em relação à aula 1 (estão aqui para quem precisar recriar a base).

Git — comandos

```
git add -A
git commit -m "chore: estado final da aula 1"   # se houver algo pendente
git switch -c lab2-auditoria
git branch
```

`src/util.js` (80 linhas)

src/util.js

```
// ============================================================
// util.js — Funções auxiliares HTTP (somente módulos nativos)
// ============================================================
const crypto = require('node:crypto');

function lerCorpo(req) {
```

```
  return new Promise((resolve, reject) => {
    let dados = '';
    req.on('data', (parte) => {
      dados += parte;
      if (dados.length > 1e6) req.destroy(); // limite de 1 MB
    });
    req.on('end', () => {
      try {
        const tipo = req.headers['content-type'] || '';
        if (tipo.includes('application/json')) return resolve(dados ? JSON.parse(dados) : {});
        resolve(Object.fromEntries(new URLSearchParams(dados)));
      } catch (e) {
        reject(e);
      }
    });
    req.on('error', reject);
  });
}

function json(res, status, obj, extras = {}) {
  res.writeHead(status, { 'Content-Type': 'application/json; charset=utf-8', 'Cache-Control': 'no-
store', ...extras });
  res.end(JSON.stringify(obj, null, 2));
}

function html(res, status, corpo, extras = {}) {
  res.writeHead(status, { 'Content-Type': 'text/html; charset=utf-8', 'Cache-Control': 'no-store',
...extras });
  res.end(corpo);
}

function redirecionar(res, url) {
  res.writeHead(302, { Location: url });
  res.end();
}

function lerCookies(req) {
  return Object.fromEntries(
    (req.headers.cookie || '').split(';').filter((c) => c.includes('=')).map((c) => {
      const i = c.indexOf('=');
      return [c.slice(0, i).trim(), decodeURIComponent(c.slice(i + 1).trim())];
    })
  );
}
```

```
const aleatorio = (bytes = 32) => crypto.randomBytes(bytes).toString('base64url');
const sha256b64url = (texto) => crypto.createHash('sha256').update(texto).digest('base64url');

function iguaisSeguro(a, b) {
  const x = Buffer.from(String(a ?? ''));
  const y = Buffer.from(String(b ?? ''));
  return x.length === y.length && crypto.timingSafeEqual(x, y);
}

const esc = (s) =>
  String(s ?? '').replace(/[&<>"']/g, (c) => ({ '&': '&amp;', '<': '&lt;', '>': '&gt;', '"':
'&quot;', "'": '&#39;' }[c]));

function pagina(titulo, corpo, cor = '#2563eb') {
  return `<!doctype html><html lang="pt-BR"><head><meta charset="utf-8">
<meta name="viewport" content="width=device-width,initial-scale=1"><title>${esc(titulo)}</title>
<style>
 body{font-family:system-ui,Segoe UI,sans-serif;max-width:860px;margin:2rem auto;padding:0
1rem;color:#1f2937;background:#f8fafc}
 .tag{display:inline-block;background:${cor};color:#fff;padding:.2rem .6rem;border-
radius:99px;font-size:.8rem}
 h1{margin:.4rem 0 1rem} h2{font-size:1.05rem;margin-top:1.6rem}
 pre{background:#0f172a;color:#e2e8f0;padding:1rem;border-radius:8px;overflow:auto;font-
size:.85rem}
 .btn,button{display:inline-block;background:${cor};color:#fff;border:0;padding:.55rem
.9rem;border-radius:6px;
   text-decoration:none;font-size:.9rem;cursor:pointer;margin:.2rem .2rem .2rem 0}
 .sec{background:#475569} .perigo{background:#b91c1c}
 form{display:inline} label{display:block;margin:.5rem 0} input{padding:.45rem;border:1px solid
#cbd5e1;border-radius:6px}
 form.bloco{display:block;background:#fff;padding:1rem;border-radius:8px;border:1px solid #e2e8f0}
 .erro{color:#b91c1c;font-weight:600} .dica{font-size:.85rem;color:#475569} ol li{margin:.25rem
0;font-size:.9rem}
</style></head><body>${corpo}</body></html>`;
}

module.exports = { lerCorpo, json, html, redirecionar, lerCookies, aleatorio, sha256b64url,
iguaisSeguro, esc, pagina };
```

`src/jwt.js` (42 linhas)

src/jwt.js

```
// ============================================================
// jwt.js — Assinatura e validação de JWT (HS256) sem bibliotecas
// Em produção prefira bibliotecas consolidadas (ex.: "jose") e
// assinatura assimétrica (RS256/ES256) publicada via JWKS.
// ============================================================
const crypto = require('node:crypto');

const b64 = (obj) => Buffer.from(JSON.stringify(obj)).toString('base64url');
const hmac = (dados, segredo) => crypto.createHmac('sha256',
segredo).update(dados).digest('base64url');

function assinar(payload, segredo) {
  const cabecalho = b64({ alg: 'HS256', typ: 'JWT' });
  const corpo = b64(payload);
  return `${cabecalho}.${corpo}.${hmac(`${cabecalho}.${corpo}`, segredo)}`;
}

function verificar(token, segredo) {
  const partes = String(token || '').split('.');
  if (partes.length !== 3) throw new Error('token malformado');
  const [cabecalho, corpo, assinatura] = partes;
  const esperado = Buffer.from(hmac(`${cabecalho}.${corpo}`, segredo));
  const recebido = Buffer.from(assinatura);
  // comparação em tempo constante (evita ataque de temporização)
  if (esperado.length !== recebido.length || !crypto.timingSafeEqual(esperado, recebido)) {
    throw new Error('assinatura invalida (token adulterado ou de outro emissor)');
  }
  const payload = JSON.parse(Buffer.from(corpo, 'base64url').toString());
  if (!payload.exp || Math.floor(Date.now() / 1000) >= payload.exp) throw new Error('token
expirado');
  return payload;
}

// Apenas lê as claims (NÃO valida) — usado pelo cliente para exibir na tela
function decodificar(token) {
  try {
    return JSON.parse(Buffer.from(String(token).split('.')[1], 'base64url').toString());
  } catch {
    return null;
  }
}

module.exports = { assinar, verificar, decodificar };
```

`src/client-credentials.js` (39 linhas)

src/client-credentials.js

```
// ============================================================
// client-credentials.js — Comunicação SERVIDOR-SERVIDOR
// O "Serviço de Relatórios" não tem usuário: autentica-se com o
// próprio client_id/client_secret e recebe um token só dele.
// Uso (com "npm start" rodando em outro terminal): npm run cc
// ============================================================
const cfg = require('./config');

const svc = cfg.CLIENTES.find((c) => c.client_id === 'servico-relatorios');
const API = `http://localhost:${cfg.PORTAS.api}`;
const basic = Buffer.from(`${svc.client_id}:${svc.client_secret}`).toString('base64');

async function chamar(caminho, token) {
  const r = await fetch(`${API}${caminho}`, { headers: { Authorization: `Bearer ${token}` } });
  console.log(`   GET ${caminho} -> HTTP ${r.status}`);
  console.log(JSON.stringify(await r.json(), null, 2).replace(/^/gm, '   '));
}

(async () => {
  console.log('1) Serviço de Relatórios pede token direto ao provedor
(grant_type=client_credentials)');
  const r = await fetch(`${cfg.EMISSOR}/token`, {
    method: 'POST',
    headers: { Authorization: `Basic ${basic}`, 'Content-Type': 'application/x-www-form-urlencoded'
},
    body: new URLSearchParams({ grant_type: 'client_credentials', scope: 'relatorios:read' }),
  });
  const t = await r.json();
  console.log(`   HTTP ${r.status} | scope=${t.scope} | expires_in=${t.expires_in}s |
refresh_token=${t.refresh_token ?? '(nenhum — não há usuário)'}`);
  if (!r.ok) process.exit(1);

  console.log('2) Usa o token no recurso permitido pelo escopo:');
  await chamar('/relatorios/resumo', t.access_token);

  console.log('3) Tenta um recurso de usuário (fora do escopo) — esperado 403:');
  await chamar('/transacoes', t.access_token);
})().catch((e) => {
  console.error(`Erro: ${e.message}. O laboratório está rodando? Execute "npm start" em outro
terminal.`);
```

```
  process.exit(1);
});
```

`testes/requisicoes.http` (32 linhas)

testes/requisicoes.http

```
# Requisições para a extensão REST Client (VS Code) — clique em "Send Request"
# Pré-requisito: "npm start" rodando.

### 1) Metadados do provedor (descoberta)
GET http://localhost:4000/.well-known/oauth-authorization-server

### 2) Client Credentials — token do Serviço de Relatórios
# @name tokenServico
POST http://localhost:4000/token
Authorization: Basic servico-relatorios:segredo-relatorios
Content-Type: application/x-www-form-urlencoded

grant_type=client_credentials&scope=relatorios:read

### 3) Relatório com o token do serviço (esperado 200)
GET http://localhost:5000/relatorios/resumo
Authorization: Bearer {{tokenServico.response.body.access_token}}

### 4) Recurso de usuário com token do serviço (esperado 403 insufficient_scope)
GET http://localhost:5000/transacoes
Authorization: Bearer {{tokenServico.response.body.access_token}}

### 5) Sem token (esperado 401 invalid_token)
GET http://localhost:5000/saldo

### 6) Segredo errado (esperado 401 invalid_client)
POST http://localhost:4000/token
Authorization: Basic servico-relatorios:segredo-errado
Content-Type: application/x-www-form-urlencoded

grant_type=client_credentials&scope=relatorios:read
```

`docs/plano-oauth.md` (57 linhas)

docs/plano-oauth.md

```
# Plano de implementação de OAuth 2.0 — API Financeira
```

Grupo: _(nomes dos integrantes)_ **Turma:** _____ **Data:** ___/___/_____

1. Contexto
A API financeira expõe saldo, extrato e transferências. Ela é consumida por (a) um aplicativo web usado por pessoas e (b) um serviço interno de relatórios que roda sem usuário. Objetivo: permitir esse acesso **sem que nenhuma aplicação conheça a senha do usuário**, com permissões mínimas e tokens de vida curta.

2. Escolha do fluxo (justificativa)
Cliente	Fluxo escolhido	Por quê
App Finanças (web, com back-end)	**Authorization Code + PKCE**	O usuário se autentica só no provedor; o código trafega pelo navegador mas só vale junto com o `code_verifier` (PKCE) e o `client_secret`, que ficam no back-end. É o fluxo recomendado pelas boas práticas atuais (RFC 9700 / OAuth 2.1).
SPA ou app mobile (futuro)	**Authorization Code + PKCE** (cliente público, sem segredo)	O **Implicit Flow** foi descontinuado: entrega o token na URL, expondo-o a histórico, logs e scripts.
Serviço de Relatórios	**Client Credentials**	Comunicação servidor-servidor: não existe usuário, a aplicação age em nome próprio. Não recebe refresh token.

3. Configuração
1. **Registro da aplicação** no provedor (no laboratório: `src/config.js`): `client_id`, `client_secret` (guardado em variável de ambiente/cofre, nunca no Git), tipo de cliente e fluxos permitidos (`grant_types`).
2. **URLs de redirecionamento**: lista exata, sempre HTTPS em produção (`https://app.exemplo.com.br/callback`), sem curingas. O provedor recusa qualquer outra — no lab, `http://site-malicioso.exemplo/callback` é bloqueada.
3. **Escopos** (menor privilégio):

Escopo	Permite	Quem recebe
`saldo:read`	Consultar saldo	App Finanças
`transacoes:read`	Consultar extrato	App Finanças
`transacoes:write`	Transferências	Somente com consentimento explícito e, idealmente, autenticação forte (MFA)
`relatorios:read`	Relatório consolidado	Serviço de Relatórios

4. **Proteções do pedido**: `state` aleatório por pedido (anti-CSRF), PKCE `S256` obrigatório, código de autorização de **uso único** com validade de 60 s.
5. **Validação na API**: assinatura do JWT, `exp`, `iss`, `aud` e escopo por rota — 401 para token inválido/expirado, 403 para escopo insuficiente.

4. Gestão de tokens

```
| Token | Formato | Validade | Onde fica |
|---|---|---|---|
| Access token | JWT assinado | 5–15 min (lab: 2 min) | Memória do back-end do cliente |
| Refresh token | Opaco (aleatório) | 30 min a alguns dias, conforme risco | Back-end do cliente (nunca em localStorage) |
| Código de autorização | Opaco | 60 s, uso único | Trafega uma vez pelo navegador |

**Exemplo prático:** às 10h00 o usuário autoriza o app e recebe access (expira 10h05) + refresh. Às 10h06 a API responde 401 → o back-end usa o refresh e recebe um **novo par** (rotação); o refresh antigo é invalidado. Se alguém reutilizar o refresh antigo, o provedor detecta o **reuso** e revoga toda a família de tokens, forçando novo login. No logout, o app chama `/revoke`.

## 5. Comunicação com a equipe
- Resumo de uma página + diagrama de sequência do fluxo; decisão registrada (ADR) no repositório.
- Checklist obrigatório em code review (PKCE, `state`, redirect exato, escopos mínimos, segredos fora do código, validação `iss/aud/exp`).
- Testes automatizados de segurança no CI (`npm test`) — o pipeline falha se uma regra for quebrada.
- Demonstração ao vivo no laboratório e canal único de dúvidas; revisão do plano a cada mudança de escopo.

## 6. Evidências do laboratório
- [ ] Print do `npm test` com 25 OK
- [ ] Print do consentimento no provedor e das claims do token
- [ ] Print do 403 (escopo) e do erro de redirect_uri
- [ ] Print da renovação e da detecção de reuso
- [ ] Saída do `npm run cc`

## Referências
- SEDUC-SP. *Descomplicando o OAuth: autorização segura para APIs* — Programação Back-End, 2ª série, 4º bimestre, semana 25, aula 1.
- [RFC 6749 — The OAuth 2.0 Authorization Framework](https://www.rfc-editor.org/rfc/rfc6749)
- [RFC 7636 — PKCE](https://www.rfc-editor.org/rfc/rfc7636)
- [RFC 9700 — Best Current Practice for OAuth 2.0 Security](https://www.rfc-editor.org/rfc/rfc9700)
```

Resultado esperado: `git branch` mostra `* lab2-auditoria`.

✔ EXEMPLO DE SUCESSO

```
Switched to a new branch 'lab2-auditoria'
* lab2-auditoria
  main
```

✘ EXEMPLO DE FALHA TÍPICA

```
fatal: a branch named 'lab2-auditoria' already exists
```

Como corrigir: A branch já existe: use git switch lab2-auditoria (sem -c).

Passo 4 Definir a política de logs (config.js e .gitignore)

VS Code

O bloco `AUDITORIA` é a **configuração de logs** do roteiro (item 2): onde gravar, tamanho para rotação, retenção, chaves de integridade e de cifra, limites de força bruta e webhook de alertas. Os logs ficam **fora do Git** (`logs/` no `.gitignore`) porque contêm dados pessoais e de segurança.

`src/config.js` (82 linhas)

src/config.js

```
// ============================================================
// config.js — "Registro da aplicação" no provedor OAuth
// Equivale ao que se cadastra no console de um provedor real
// (Google Cloud, Microsoft Entra, Auth0...): client_id, segredo,
// URLs de redirecionamento permitidas, fluxos e escopos.
// ============================================================
const PORTAS = {
  auth: Number(process.env.AUTH_PORT || 4000),      // Servidor de Autorização
  api: Number(process.env.API_PORT || 5000),        // Servidor de Recursos (API financeira)
  cliente: Number(process.env.CLIENT_PORT || 3000), // Aplicação cliente (App Finanças)
};

module.exports = {
  PORTAS,
  EMISSOR: `http://localhost:${PORTAS.auth}`,
  AUDIENCIA_API: 'api-financeira',
  // Em produção: segredo longo, guardado em variável de ambiente/cofre (NUNCA no Git)
  SEGREDO_ASSINATURA: process.env.OAUTH_SEGREDO || 'troque-este-segredo-em-producao',

  // Tempos de vida (segundos) — tokens de acesso curtos, refresh mais longo
  ACCESS_TOKEN_TTL: Number(process.env.ACCESS_TTL || 120), // 2 min (didático; real: 5 a 15 min)
  REFRESH_TOKEN_TTL: Number(process.env.REFRESH_TTL || 1800), // 30 min
  CODE_TTL: 60, // código de autorização: uso único, vale 60 s

  // Escopos = permissões granulares oferecidas pela API
  ESCOPOS: {
    'saldo:read': 'Consultar saldo da conta',
    'transacoes:read': 'Consultar extrato de transações',
```

```
    'transacoes:write': 'Criar transferências (operação sensível)',
    'relatorios:read': 'Gerar relatórios consolidados (servidor-servidor)',
  },

  // Aplicações registradas no provedor
  CLIENTES: [
    {
      client_id: 'app-financas-web',
      client_secret: 'segredo-app-web', // cliente confidencial: o segredo fica só no back-end
      nome: 'App Finanças (Web)',
      redirect_uris: [`http://localhost:${PORTAS.cliente}/callback`], // lista exata (sem curingas)
      grant_types: ['authorization_code', 'refresh_token'],
      escopos_permitidos: ['saldo:read', 'transacoes:read', 'transacoes:write'],
    },
    {
      client_id: 'servico-relatorios',
      client_secret: 'segredo-relatorios',
      nome: 'Serviço de Relatórios (back-end)',
      redirect_uris: [], // sem usuário, sem redirecionamento
      grant_types: ['client_credentials'],
      escopos_permitidos: ['relatorios:read'],
    },
  ],

  // ============================================================
  // AUDITORIA E LOGS (Laboratório 2 — aula 3)
  // ============================================================
  AUDITORIA: {
    LOG_DIR: process.env.LOG_DIR || require('node:path').join(__dirname, '..', 'logs'),
    // Rotação: ao passar deste tamanho o arquivo é renomeado (auditoria-<data>-NNN.log)
    LOG_MAX_BYTES: Number(process.env.LOG_MAX_BYTES || 256 * 1024), // 256 KB (didático)
    // Retenção: mantém no máximo N arquivos rotacionados e apaga os mais antigos que X dias
    LOG_RETENCAO_ARQUIVOS: Number(process.env.LOG_RETENCAO_ARQUIVOS || 5),
    LOG_RETENCAO_DIAS: Number(process.env.LOG_RETENCAO_DIAS || 30),
    // Integridade: cada linha leva um HMAC encadeado com a anterior (detecta edição/remoção)
    CHAVE_INTEGRIDADE: process.env.LOG_CHAVE_INTEGRIDADE || 'chave-integridade-de-laboratorio',
    // Confidencialidade: dados sensíveis (ex.: login digitado errado) vão cifrados com AES-256-GCM
    CHAVE_CIFRA: process.env.LOG_CHAVE_CIFRA || 'chave-cifra-de-laboratorio',
    // Detecção de força bruta: N falhas de autenticação do mesmo IP em J segundos => alerta +
bloqueio
    EVENTOS_FALHA_AUTH: ['LOGIN_FALHA', 'CLIENTE_AUTH_FALHA'],
    LIMITE_FALHAS: Number(process.env.LIMITE_FALHAS || 5),
    JANELA_S: Number(process.env.JANELA_S || 60),
    BLOQUEIO_S: Number(process.env.BLOQUEIO_S || 300),
```

```
    // X-Forwarded-For só é confiável atrás de um proxy reverso SEU. No laboratório fica ligado
    // para simular IPs diferentes; em produção, desligue se a API estiver exposta diretamente.
    CONFIAR_PROXY: process.env.CONFIAR_PROXY !== '0',
    // Opcional: URL de webhook (ex.: Slack/Teams) que recebe os alertas críticos
    ALERTA_WEBHOOK: process.env.ALERTA_WEBHOOK || '',
  },

  // Usuário de demonstração (no provedor, nunca no cliente)
  USUARIOS: [{ id: 'u1', login: 'aluno', senha: 'senha123', nome: 'Aluno(a) Demo' }],
};
```

`.gitignore` (8 linhas)

.gitignore

```
node_modules/
.env
*.log
logs/
relatorio/
.servidor.pid
relatorio-preparacao.csv
```

Resultado esperado: Você identifica LOG_MAX_BYTES, LOG_RETENCAO_*, LIMITE_FALHAS, JANELA_S e BLOQUEIO_S.

✔ EXEMPLO DE SUCESSO

```
node -e "console.log(require('./src/config').AUDITORIA.LIMITE_FALHAS)"
5
```

✘ EXEMPLO DE FALHA TÍPICA

```
TypeError: Cannot read properties of undefined (reading 'LIMITE_FALHAS')
```

Como corrigir: O bloco AUDITORIA não foi colado dentro do module.exports de config.js.

Passo 5 Criar o módulo de auditoria

VS Code

Coração da seção: `registrar()` grava um evento JSON por linha; `mascarar()` troca segredos por impressão digital; `cifrar()` protege o login digitado; cada linha leva `hash_anterior` e `hash` (HMAC encadeado); `rotacionarSeNecessario()` e `aplicarRetencao()` controlam o armazenamento; `avaliarRegras()` dispara alertas e bloqueia IPs; `envolver()` gera o log de acesso e o `X-Request-Id`.

`src/auditoria.js` (250 linhas)

src/auditoria.js

```
// ============================================================
// auditoria.js — LOG DE AUDITORIA estruturado e seguro
//  * JSON Lines (1 evento por linha) — formato aceito por ELK,
//    Splunk, Graylog, Loki... sem conversão
//  * Mascaramento de segredos (senha, tokens, client_secret)
//  * Cifra AES-256-GCM para dados sensíveis (login digitado)
//  * Integridade: HMAC encadeado linha a linha
//  * Rotação por tamanho + retenção por quantidade e idade
//  * Regras de alerta: força bruta (com bloqueio de IP) e
//    eventos CRÍTICOS (console, alertas.log e webhook opcional)
// ============================================================
const fs = require('node:fs');
const path = require('node:path');
const crypto = require('node:crypto');
const cfg = require('./config');

const A = cfg.AUDITORIA;
const arqAtual = (dir = A.LOG_DIR) => path.join(dir, 'auditoria.log');
const arqAlertas = () => path.join(A.LOG_DIR, 'alertas.log');
const PADRAO_ROTACIONADO = /^auditoria-\d{8}T\d{9}-\d{3}\.log$/;
const CHAVE_AES = crypto.createHash('sha256').update(A.CHAVE_CIFRA).digest(); // produção: chave
vinda de cofre/KMS

let ultimoHash = null; // último elo da cadeia de integridade
const memoria = []; // últimos eventos em memória (consultas rápidas e testes)
const falhasPorIp = new Map(); // ip -> [timestamps] (janela deslizante)
const bloqueados = new Map(); // ip -> expira em (ms)

// ------------------------------------------------------------- utilidades
const hmac = (anterior, corpo) => crypto.createHmac('sha256',
A.CHAVE_INTEGRIDADE).update(`${anterior}|${corpo}`).digest('hex');
const impressao = (v) =>
`sha256:${crypto.createHash('sha256').update(String(v)).digest('hex').slice(0, 12)}`;

function cifrar(texto) {
  const iv = crypto.randomBytes(12);
  const c = crypto.createCipheriv('aes-256-gcm', CHAVE_AES, iv);
  const ct = Buffer.concat([c.update(String(texto), 'utf8'), c.final()]);
  return
`gcm:${iv.toString('base64url')}.${ct.toString('base64url')}.${c.getAuthTag().toString('base64url')
}`;
}
```

```
function decifrar(valor) {
  const [iv, ct, tag] = String(valor).slice(4).split('.').map((x) => Buffer.from(x, 'base64url'));
  const d = crypto.createDecipheriv('aes-256-gcm', CHAVE_AES, iv);
  d.setAuthTag(tag);
  return Buffer.concat([d.update(ct), d.final()]).toString('utf8');
}

// Campos cujo NOME indica segredo nunca são gravados em claro:
// strings viram uma impressão digital (permite correlacionar sem expor)
const SENSIVEL = /(senha|password|secret|authorization|code_verifier|^code$|token)/i;
function mascarar(valor, chave = '') {
  if (SENSIVEL.test(chave)) return typeof valor === 'string' ? impressao(valor) : '[OCULTO]';
  if (Array.isArray(valor)) return valor.map((v) => mascarar(v));
  if (valor && typeof valor === 'object') {
    return Object.fromEntries(Object.entries(valor).map(([k, v]) => [k, mascarar(v, k)]));
  }
  return valor;
}

// IP do cliente. X-Forwarded-For pode ser forjado: só use atrás de proxy confiável
function ipDe(req) {
  const xff = req.headers['x-forwarded-for'];
  if (A.CONFIAR_PROXY && xff) return String(xff).split(',')[0].trim().slice(0, 45);
  return String(req.socket?.remoteAddress || '').replace(/^::ffff:/, '');
}

// ---------------------------------------------------------------- arquivos
function listarArquivos(dir = A.LOG_DIR) {
  if (!fs.existsSync(dir)) return [];
  const lista = fs.readdirSync(dir).filter((f) => PADRAO_ROTACIONADO.test(f)).sort().map((f) =>
path.join(dir, f));
  if (fs.existsSync(arqAtual(dir))) lista.push(arqAtual(dir));
  return lista; // ordem cronológica: rotacionados (mais antigo primeiro) + atual
}

function lerUltimoHash() {
  const arquivos = listarArquivos();
  for (let i = arquivos.length - 1; i >= 0; i--) {
    const linhas = fs.readFileSync(arquivos[i], 'utf8').split('\n').filter(Boolean);
    if (linhas.length) {
      try { return JSON.parse(linhas[linhas.length - 1]).hash || 'SEM-HASH'; } catch { return
'LINHA-CORROMPIDA'; }
    }
```

```
  }
  return 'GENESIS';
}

function garantirInicio() {
  if (ultimoHash !== null) return;
  fs.mkdirSync(A.LOG_DIR, { recursive: true });
  ultimoHash = lerUltimoHash(); // continua a cadeia após reiniciar o servidor
}

function aplicarRetencao() {
  const rotacionados = listarArquivos().filter((f) => PADRAO_ROTACIONADO.test(path.basename(f)));
  const limiteIdade = Date.now() - A.LOG_RETENCAO_DIAS * 86400000;
  const expurgados = [];
  rotacionados.forEach((f, i) => {
    const excedeQuantidade = i < rotacionados.length - A.LOG_RETENCAO_ARQUIVOS;
    if (excedeQuantidade || fs.statSync(f).mtimeMs < limiteIdade) {
      fs.unlinkSync(f);
      expurgados.push(path.basename(f));
    }
  });
  return expurgados;
}

// Renomeia o arquivo atual quando ele passaria do limite. Devolve a informação da
// rotação, que é gravada DENTRO do próximo evento (a própria rotação fica auditada).
function rotacionarSeNecessario(bytesNovos) {
  const atual = arqAtual();
  if (!fs.existsSync(atual) || fs.statSync(atual).size + bytesNovos <= A.LOG_MAX_BYTES) return
null;
  const ts = new Date().toISOString().replace(/[-:.]/g, '').slice(0, 18); // 20261102T190512345
  let n = 0;
  let destino;
  do { destino = path.join(A.LOG_DIR, `auditoria-${ts}-${String(n++).padStart(3, '0')}.log`); }
while (fs.existsSync(destino));
  try {
    fs.renameSync(atual, destino); // no Windows falha se outro programa travar o arquivo
  } catch (e) {
    console.error(`[auditoria] rotação adiada: ${e.message}`);
    return null;
  }
  return { arquivo_rotacionado: path.basename(destino), expurgados: aplicarRetencao() };
}
```

```
// ------------------------------------------------------------- registro
function registrar(dados) {
  garantirInicio();
  const { req, ...resto } = dados;
  const registro = { ts: new Date().toISOString(), nivel: 'INFO', ...mascarar(resto) };
  if (req) {
    if (!registro.ip) registro.ip = req.ipCliente || ipDe(req);
    if (!registro.req_id && req.idReq) registro.req_id = req.idReq;
  }
  const montar = () => {
    registro.hash_anterior = ultimoHash;
    delete registro.hash;
    registro.hash = hmac(ultimoHash, JSON.stringify(registro));
    return `${JSON.stringify(registro)}\n`;
  };
  let linha = montar();
  const rotacao = rotacionarSeNecessario(Buffer.byteLength(linha));
  if (rotacao) {
    delete registro.hash_anterior;
    registro.rotacao = rotacao;
    linha = montar();
  }
  fs.appendFileSync(arqAtual(), linha, { mode: 0o600 }); // síncrono: ordem garantida
  ultimoHash = registro.hash;
  memoria.push(registro);
  if (memoria.length > 2000) memoria.shift();
  avaliarRegras(registro);
  return registro;
}

// ------------------------------------------------------------- regras / alertas
function estaBloqueado(ip) {
  const ate = bloqueados.get(ip);
  if (!ate) return false;
  if (Date.now() > ate) { bloqueados.delete(ip); return false; }
  return true;
}

function emitirAlerta(r) {
  const texto = `[ALERTA ${r.nivel}] ${r.ts} ${r.tipo} ip=${r.ip || '-'} ${r.motivo || r.acao ||
''}`;
  if (!process.env.OAUTH_QUIET) console.error(`\x1b[41m\x1b[97m${texto}\x1b[0m`);
  fs.appendFileSync(arqAlertas(), `${JSON.stringify(r)}\n`, { mode: 0o600 });
  if (A.ALERTA_WEBHOOK) {
```

```
    fetch(A.ALERTA_WEBHOOK, { method: 'POST', headers: { 'Content-Type': 'application/json' },
body: JSON.stringify({ text: texto }) })
      .catch((e) => console.error(`[auditoria] webhook falhou: ${e.message}`));
  }
}

function avaliarRegras(r) {
  if (r.nivel === 'CRITICO') emitirAlerta(r);
  if (!A.EVENTOS_FALHA_AUTH.includes(r.tipo) || !r.ip) return;
  const agora = Date.now();
  const janela = (falhasPorIp.get(r.ip) || []).filter((t) => agora - t < A.JANELA_S * 1000);
  janela.push(agora);
  falhasPorIp.set(r.ip, janela);
  if (janela.length >= A.LIMITE_FALHAS && !estaBloqueado(r.ip)) {
    bloqueados.set(r.ip, agora + A.BLOQUEIO_S * 1000);
    falhasPorIp.delete(r.ip);
    registrar({ tipo: 'ALERTA_FORCA_BRUTA', nivel: 'CRITICO', origem: r.origem, ip: r.ip, req_id:
r.req_id, falhas: janela.length, janela_s: A.JANELA_S, acao: `IP bloqueado por ${A.BLOQUEIO_S}s`
});
  }
}

// ------------------------------------------------------------------ middleware HTTP
// Envolve o handler: gera/propaga X-Request-Id, mede a duração e grava o log de acesso.
// A query string NÃO é gravada (pode conter code, state e outros dados sensíveis).
function envolver(origem, handler) {
  return (req, res) => {
    const recebido = String(req.headers['x-request-id'] || '');
    req.idReq = /^[A-Za-z0-9._-]{8,64}$/.test(recebido) ? recebido : crypto.randomUUID(); // evita
injeção no log
    req.ipCliente = ipDe(req);
    res.setHeader('X-Request-Id', req.idReq);
    const inicio = process.hrtime.bigint();
    res.on('finish', () => {
      const status = res.statusCode;
      registrar({
        tipo: 'HTTP_REQUISICAO', nivel: status >= 500 ? 'ERRO' : status >= 400 ? 'AVISO' : 'INFO',
origem,
        metodo: req.method, rota: new URL(req.url, 'http://local').pathname, status,
        duracao_ms: Math.round(Number(process.hrtime.bigint() - inicio) / 1e4) / 100,
        user_agent: String(req.headers['user-agent'] || '').slice(0, 120), req,
      });
    });
    return handler(req, res);
```

```
  };
}

// ------------------------------------------------------------- verificação
function verificarIntegridade(dir = A.LOG_DIR) {
  const arquivos = listarArquivos(dir);
  const erros = [];
  let anterior = null;
  let total = 0;
  let ancora = null;
  for (const arq of arquivos) {
    const nome = path.basename(arq);
    fs.readFileSync(arq, 'utf8').split('\n').forEach((linha, i) => {
      if (!linha.trim()) return;
      total++;
      let r;
      try { r = JSON.parse(linha); } catch {
        erros.push({ arquivo: nome, linha: i + 1, motivo: 'linha não é JSON válido' });
        anterior = null;
        return;
      }
      const { hash, ...corpo } = r;
      if (anterior === null && !ancora) ancora = { arquivo: nome, linha: i + 1, completa:
r.hash_anterior === 'GENESIS' };
      else if (anterior !== null && r.hash_anterior !== anterior) erros.push({ arquivo: nome,
linha: i + 1, motivo: 'encadeamento quebrado (linha removida, inserida ou reordenada)' });
      if (hmac(r.hash_anterior, JSON.stringify(corpo)) !== hash) erros.push({ arquivo: nome, linha:
i + 1, motivo: 'conteúdo alterado (HMAC não confere)' });
      anterior = hash;
    });
  }
  return { ok: erros.length === 0, total, arquivos: arquivos.map((a) => path.basename(a)), ancora,
erros };
}

function lerEventos(dir = A.LOG_DIR) {
  const eventos = [];
  for (const arq of listarArquivos(dir)) {
    for (const linha of fs.readFileSync(arq, 'utf8').split('\n')) {
      if (!linha.trim()) continue;
      try { eventos.push(JSON.parse(linha)); } catch { /* linha corrompida: apontada pela
verificação */ }
    }
  }
```

```
  return eventos;
}

module.exports = {
  registrar, envolver, estaBloqueado, cifrar, decifrar, impressao, mascarar,
  verificarIntegridade, lerEventos, listarArquivos, memoria,
};
```

Resultado esperado: Você localiza onde a query string deixa de ser gravada e por quê.

✔ EXEMPLO DE SUCESSO

```
node --check src\auditoria.js   (sem saída)
```

✘ EXEMPLO DE FALHA TÍPICA

```
Error: Cannot find module './config'
```

Como corrigir: auditoria.js precisa estar em src\, ao lado de config.js.

Passo 6 Instrumentar o Servidor de Autorização

VS Code

Novos eventos: `LOGIN_SUCESSO`, `LOGIN_FALHA` (login cifrado), `CLIENTE_AUTH_FALHA`, `TOKEN_EMITIDO` (com `jti`), `TOKEN_RECUSADO`, `REFRESH_REUSO` (CRÍTICO), `TOKEN_REVOGADO`, `AUTORIZACAO_RECUSADA`. IP bloqueado recebe **429** com `Retry-After`. Erros 500 não expõem detalhes ao cliente — só ao log.

`src/auth-server.js` (299 linhas)

src/auth-server.js

```
// ============================================================
// auth-server.js — SERVIDOR DE AUTORIZAÇÃO (porta 4000)
// Implementa: Authorization Code + PKCE, Refresh Token (com
// rotação e detecção de reuso), Client Credentials e revogação.
// Lab 2: registra eventos de auditoria e bloqueia IPs com
// excesso de falhas de autenticação (força bruta).
// ============================================================
const http = require('node:http');
const cfg = require('./config');
const jwt = require('./jwt');
const { lerCorpo, json, html, redirecionar, aleatorio, sha256b64url, iguaisSeguro, esc, pagina } =
require('./util');
const auditoria = require('./auditoria');
```

```
const audit = (req, tipo, extra = {}) => auditoria.registrar({ origem: 'auth', tipo, req, ...extra
});

// IP em bloqueio temporário (força bruta): responde 429 sem processar o pedido
function recusarSeBloqueado(req, res, comoHtml = false) {
  if (!auditoria.estaBloqueado(req.ipCliente)) return false;
  audit(req, 'REQUISICAO_BLOQUEADA', { nivel: 'AVISO', rota: new URL(req.url,
cfg.EMISSOR).pathname, motivo: 'IP em bloqueio temporário por força bruta' });
  const extras = { 'Retry-After': String(cfg.AUDITORIA.BLOQUEIO_S) };
  if (comoHtml) html(res, 429, paginaErro('Muitas tentativas de login falharam a partir deste
endereço. Aguarde alguns minutos e tente de novo.'), extras);
  else json(res, 429, { error: 'too_many_requests', error_description: 'Muitas falhas de
autenticação deste IP. Tente mais tarde.' }, extras);
  return true;
}

// client_id que o chamador AFIRMA ser (para registrar falhas de autenticação do cliente)
function idInformado(req, corpo) {
  const auth = req.headers.authorization || '';
  if (auth.startsWith('Basic ')) return decodeURIComponent(Buffer.from(auth.slice(6),
'base64').toString().split(':')[0] || '').slice(0, 80);
  return String(corpo.client_id || '').slice(0, 80);
}

const codigos = new Map(); // code -> { client_id, redirect_uri, escopos, sub, code_challenge, exp
}
const refreshTokens = new Map(); // refresh -> { sub, client_id, escopos, exp, familia }
const refreshUsados = new Map(); // refresh já trocado -> familia (para detectar reuso)

const agora = () => Math.floor(Date.now() / 1000);
const buscarCliente = (id) => cfg.CLIENTES.find((c) => c.client_id === id);
const log = (...a) => { if (!process.env.OAUTH_QUIET) console.log('[auth]', ...a); };

function validarEscopo(cliente, scope) {
  const pedidos = String(scope || '').split(' ').filter(Boolean);
  if (!pedidos.length) return null;
  return pedidos.every((e) => cliente.escopos_permitidos.includes(e)) ? pedidos : null;
}

function autenticarCliente(req, corpo) {
  let id = corpo.client_id;
  let segredo = corpo.client_secret;
  const auth = req.headers.authorization || '';
  if (auth.startsWith('Basic ')) {
```

```
    const [i, s] = Buffer.from(auth.slice(6), 'base64').toString().split(':');
    id = decodeURIComponent(i || '');
    segredo = decodeURIComponent(s || '');
  }
  const cliente = buscarCliente(id);
  return cliente && iguaisSeguro(cliente.client_secret, segredo) ? cliente : null;
}

function emitirAccessToken(sub, client_id, escopos) {
  const iat = agora();
  return jwt.assinar(
    { iss: cfg.EMISSOR, aud: cfg.AUDIENCIA_API, sub, client_id, scope: escopos.join(' '), iat, exp:
iat + cfg.ACCESS_TOKEN_TTL, jti: aleatorio(8) },
    cfg.SEGREDO_ASSINATURA
  );
}

function emitirTokens(sub, client_id, escopos, comRefresh, familia = aleatorio(8)) {
  const r = { access_token: emitirAccessToken(sub, client_id, escopos), token_type: 'Bearer',
expires_in: cfg.ACCESS_TOKEN_TTL, scope: escopos.join(' ') };
  if (comRefresh) {
    r.refresh_token = aleatorio(32); // opaco: só o provedor sabe o que significa
    r.refresh_expires_in = cfg.REFRESH_TOKEN_TTL;
    refreshTokens.set(r.refresh_token, { sub, client_id, escopos, exp: agora() +
cfg.REFRESH_TOKEN_TTL, familia });
  }
  return r;
}

function revogarFamilia(familia) {
  for (const [token, d] of refreshTokens) if (d.familia === familia) refreshTokens.delete(token);
}

// ---------- /authorize ------------------------------------------------
function validarPedido(p) {
  const cliente = buscarCliente(p.client_id);
  if (!cliente) return { fatal: 'client_id desconhecido: a aplicação não está registrada no
provedor.' };
  if (!cliente.redirect_uris.includes(p.redirect_uri)) {
    return { fatal: `redirect_uri não registrada para "${cliente.client_id}": ${p.redirect_uri}.
Por segurança o provedor NÃO redireciona para URLs desconhecidas (evita roubo do código).` };
  }
  const erro = (error, error_description) => ({ cliente, erro: { error, error_description } });
  if (!cliente.grant_types.includes('authorization_code')) return erro('unauthorized_client',
```

```
'Cliente não habilitado para o Authorization Code Flow.');
  if (p.response_type !== 'code') return erro('unsupported_response_type', 'Use response_type=code.
O Implicit Flow (response_type=token) é desaconselhado pelas boas práticas atuais.');
  const escopos = validarEscopo(cliente, p.scope);
  if (!escopos) return erro('invalid_scope', 'Escopo ausente ou não permitido para este cliente.');
  if (!p.code_challenge || p.code_challenge_method !== 'S256') return erro('invalid_request', 'PKCE
obrigatório: envie code_challenge com code_challenge_method=S256.');
  if (!p.state) return erro('invalid_request', 'Parâmetro state obrigatório (proteção contra
CSRF).');
  return { cliente, escopos };
}

function voltarComErro(res, p, erro) {
  const u = new URL(p.redirect_uri);
  u.searchParams.set('error', erro.error);
  u.searchParams.set('error_description', erro.error_description);
  if (p.state) u.searchParams.set('state', p.state);
  redirecionar(res, u.toString());
}

function paginaErro(msg) {
  return pagina('Provedor OAuth — erro', `<p class="tag">Servidor de Autorização · porta
${cfg.PORTAS.auth}</p>
  <h1>Pedido de autorização recusado</h1><p class="erro">${esc(msg)}</p>
  <a class="btn sec" href="http://localhost:${cfg.PORTAS.cliente}/">Voltar ao App Finanças</a>`,
'#7c3aed');
}

function telaConsentimento(p, cliente, escopos, mensagem = '') {
  const campos = ['client_id', 'redirect_uri', 'response_type', 'scope', 'state', 'code_challenge',
'code_challenge_method']
    .map((k) => `<input type="hidden" name="${k}" value="${esc(p[k])}">`).join('');
  return pagina('Provedor OAuth — login e consentimento', `
  <p class="tag">Servidor de Autorização · porta ${cfg.PORTAS.auth}</p>
  <h1>Entrar no Provedor OAuth</h1>
  <p>A aplicação <b>${esc(cliente.nome)}</b> (<code>${esc(cliente.client_id)}</code>) solicita
acesso a:</p>
  <ul>${escopos.map((e) => `<li><code>${esc(e)}</code> — ${esc(cfg.ESCOPOS[e] || '')}
</li>`).join('')}</ul>
  ${mensagem ? `<p class="erro">${esc(mensagem)}</p>` : ''}
  <form method="post" action="/authorize" class="bloco">${campos}
    <label>Usuário <input name="login" value="aluno" autocomplete="username"></label>
    <label>Senha <input name="senha" type="password" placeholder="senha123" autocomplete="current-
password"></label>
```

```
    <p class="dica">A senha é digitada <b>somente aqui, no provedor</b>. O App Finanças nunca a
recebe.</p>
    <button name="decisao" value="permitir">Permitir acesso</button>
    <button name="decisao" value="negar" class="sec">Negar</button>
  </form>`, '#7c3aed');
}

function registrarRecusa(req, p, v) {
  audit(req, 'AUTORIZACAO_RECUSADA', { nivel: 'AVISO', client_id: String(p.client_id ||
'').slice(0, 80), redirect_uri: String(p.redirect_uri || '').slice(0, 200), motivo: v.fatal ?
'redirect_uri ou client_id não registrados' : v.erro.error });
}

function getAuthorize(req, res, url) {
  const p = Object.fromEntries(url.searchParams);
  const v = validarPedido(p);
  if (v.fatal || v.erro) registrarRecusa(req, p, v);
  if (v.fatal) return html(res, 400, paginaErro(v.fatal));
  if (v.erro) return voltarComErro(res, p, v.erro);
  html(res, 200, telaConsentimento(p, v.cliente, v.escopos));
}

async function postAuthorize(req, res) {
  if (recusarSeBloqueado(req, res, true)) return;
  const p = await lerCorpo(req);
  const v = validarPedido(p);
  if (v.fatal || v.erro) registrarRecusa(req, p, v);
  if (v.fatal) return html(res, 400, paginaErro(v.fatal));
  if (v.erro) return voltarComErro(res, p, v.erro);
  if (p.decisao === 'negar') {
    audit(req, 'CONSENTIMENTO_NEGADO', { client_id: p.client_id });
    return voltarComErro(res, p, { error: 'access_denied', error_description: 'O usuário negou o
acesso.' });
  }
  const usuario = cfg.USUARIOS.find((u) => u.login === p.login && iguaisSeguro(u.senha, p.senha));
  if (!usuario) {
    // o login digitado vai CIFRADO: usuários às vezes digitam a senha no campo de login
    audit(req, 'LOGIN_FALHA', { nivel: 'AVISO', client_id: p.client_id, login_cifrado:
auditoria.cifrar(String(p.login || '').slice(0, 100)) });
    return html(res, 401, telaConsentimento(p, v.cliente, v.escopos, 'Usuário ou senha
inválidos.'));
  }
  audit(req, 'LOGIN_SUCESSO', { sub: usuario.id, client_id: p.client_id, escopos: v.escopos });
  const code = aleatorio(24);
```

```
  codigos.set(code, { client_id: p.client_id, redirect_uri: p.redirect_uri, escopos: v.escopos,
sub: usuario.id, code_challenge: p.code_challenge, exp: agora() + cfg.CODE_TTL });
  log(`código emitido para ${p.client_id} (usuário ${usuario.login}, escopos: ${v.escopos.join('
')})`);
  const u = new URL(p.redirect_uri);
  u.searchParams.set('code', code);
  u.searchParams.set('state', p.state);
  redirecionar(res, u.toString());
}

// ---------- /token ----------------------------------------------------
function responderTokens(req, res, grant, sub, cliente, escopos, comRefresh, familia) {
  const tokens = emitirTokens(sub, cliente.client_id, escopos, comRefresh, familia);
  audit(req, 'TOKEN_EMITIDO', { grant_type: grant, client_id: cliente.client_id, sub, scope:
tokens.scope, jti: jwt.decodificar(tokens.access_token).jti });
  return json(res, 200, tokens);
}

async function postToken(req, res) {
  if (recusarSeBloqueado(req, res)) return;
  const corpo = await lerCorpo(req);
  const grant = String(corpo.grant_type || '').slice(0, 40);
  const recusar = (status, error, d, nivel = 'AVISO') => {
    audit(req, 'TOKEN_RECUSADO', { nivel, grant_type: grant, client_id: idInformado(req, corpo),
erro: error, motivo: d });
    return json(res, status, { error, error_description: d });
  };
  const erroGrant = (d) => recusar(400, 'invalid_grant', d);
  const suportados = ['authorization_code', 'refresh_token', 'client_credentials'];
  if (!suportados.includes(grant)) return recusar(400, 'unsupported_grant_type', `Use:
${suportados.join(', ')}`);

  const cliente = autenticarCliente(req, corpo);
  if (!cliente) {
    audit(req, 'CLIENTE_AUTH_FALHA', { nivel: 'AVISO', client_id: idInformado(req, corpo),
grant_type: grant });
    return json(res, 401, { error: 'invalid_client', error_description: 'Falha na autenticação do
cliente (client_id/client_secret).' }, { 'WWW-Authenticate': 'Basic realm="oauth"' });
  }
  if (!cliente.grant_types.includes(grant)) return recusar(400, 'unauthorized_client', `Cliente não
habilitado para ${grant}.`);

  if (corpo.grant_type === 'authorization_code') {
    const d = codigos.get(corpo.code);
```

```
    codigos.delete(corpo.code); // uso único — descartado mesmo se a troca falhar
    if (!d || d.exp < agora()) return erroGrant('Código inválido, expirado ou já utilizado.');
    if (d.client_id !== cliente.client_id) return erroGrant('Código emitido para outro cliente.');
    if (d.redirect_uri !== corpo.redirect_uri) return erroGrant('redirect_uri diferente da usada na
autorização.');
    if (!corpo.code_verifier || sha256b64url(corpo.code_verifier) !== d.code_challenge) return
erroGrant('PKCE: code_verifier não confere com o code_challenge.');
    log(`tokens emitidos (authorization_code) para ${cliente.client_id}`);
    return responderTokens(req, res, grant, d.sub, cliente, d.escopos, true);
  }

  if (corpo.grant_type === 'refresh_token') {
    const rt = corpo.refresh_token;
    if (refreshUsados.has(rt)) {
      revogarFamilia(refreshUsados.get(rt));
      log('ALERTA: reuso de refresh token — família revogada');
      audit(req, 'REFRESH_REUSO', { nivel: 'CRITICO', client_id: cliente.client_id, refresh_token:
rt, motivo: 'refresh token já trocado foi reapresentado (possível roubo); família revogada' });
      return erroGrant('REUSO de refresh token detectado: todos os tokens desta sessão foram
revogados. Faça login novamente.');
    }
    const d = refreshTokens.get(rt);
    if (!d || d.client_id !== cliente.client_id) return erroGrant('Refresh token inválido ou
revogado.');
    if (d.exp < agora()) {
      refreshTokens.delete(rt);
      return erroGrant('Refresh token expirado: faça login novamente.');
    }
    refreshTokens.delete(rt); // ROTAÇÃO: o antigo deixa de valer...
    refreshUsados.set(rt, d.familia); // ...e passa a ser vigiado
    log(`tokens renovados (refresh_token) para ${cliente.client_id}`);
    return responderTokens(req, res, grant, d.sub, cliente, d.escopos, true, d.familia);
  }

  // client_credentials: a aplicação age em nome PRÓPRIO (sem usuário, sem refresh)
  const escopos = validarEscopo(cliente, corpo.scope || cliente.escopos_permitidos.join(' '));
  if (!escopos) return recusar(400, 'invalid_scope', 'Escopo não permitido para este cliente.');
  log(`token emitido (client_credentials) para ${cliente.client_id}`);
  return responderTokens(req, res, grant, cliente.client_id, cliente, escopos, false);
}

// ---------- /revoke (RFC 7009) ----------------------------------------
async function postRevoke(req, res) {
  const corpo = await lerCorpo(req);
```

```
  const cliente = autenticarCliente(req, corpo);
  if (!cliente) {
    audit(req, 'CLIENTE_AUTH_FALHA', { nivel: 'AVISO', client_id: idInformado(req, corpo),
grant_type: 'revoke' });
    return json(res, 401, { error: 'invalid_client' });
  }
  const d = refreshTokens.get(corpo.token);
  if (d) revogarFamilia(d.familia);
  audit(req, 'TOKEN_REVOGADO', { client_id: cliente.client_id, sub: d?.sub, encontrado: Boolean(d)
});
  json(res, 200, { revogado: true }); // responde 200 mesmo se o token não existir
}

function metadados() {
  const e = cfg.EMISSOR;
  return {
    issuer: e,
    authorization_endpoint: `${e}/authorize`,
    token_endpoint: `${e}/token`,
    revocation_endpoint: `${e}/revoke`,
    response_types_supported: ['code'],
    grant_types_supported: ['authorization_code', 'refresh_token', 'client_credentials'],
    code_challenge_methods_supported: ['S256'],
    token_endpoint_auth_methods_supported: ['client_secret_basic', 'client_secret_post'],
    scopes_supported: Object.keys(cfg.ESCOPOS),
  };
}

function criarServidor() {
  return http.createServer(auditoria.envolver('auth', async (req, res) => {
    const url = new URL(req.url, cfg.EMISSOR);
    log(req.method, url.pathname);
    try {
      if (req.method === 'GET' && url.pathname === '/authorize') return getAuthorize(req, res,
url);
      if (req.method === 'POST' && url.pathname === '/authorize') return await postAuthorize(req,
res);
      if (req.method === 'POST' && url.pathname === '/token') return await postToken(req, res);
      if (req.method === 'POST' && url.pathname === '/revoke') return await postRevoke(req, res);
      if (req.method === 'GET' && url.pathname === '/.well-known/oauth-authorization-server')
return json(res, 200, metadados());
      if (req.method === 'GET' && url.pathname === '/') {
        const lista = cfg.CLIENTES.map((c) => ({ client_id: c.client_id, nome: c.nome,
redirect_uris: c.redirect_uris, grant_types: c.grant_types, escopos_permitidos:
```

```
c.escopos_permitidos }));
        return html(res, 200, pagina('Provedor OAuth', `<p class="tag">Servidor de Autorização · porta ${cfg.PORTAS.auth}</p>
          <h1>Provedor OAuth (didático)</h1><h2>Aplicações registradas (segredos ocultos)</h2><pre>${esc(JSON.stringify(lista, null, 2))}</pre>
          <h2>Metadados</h2><pre>${esc(JSON.stringify(metadados(), null, 2))}</pre>`, '#7c3aed'));
      }
      json(res, 404, { error: 'not_found' });
    } catch (e) {
      console.error('[auth] erro:', e);
      audit(req, 'ERRO_SERVIDOR', { nivel: 'ERRO', motivo: e.message });
      json(res, 500, { error: 'server_error', error_description: 'Erro interno.' }); // detalhe só no log
    }
  }));
}

function iniciar(porta = cfg.PORTAS.auth) {
  return new Promise((resolve, reject) => {
    const s = criarServidor();
    s.once('error', reject);
    s.listen(porta, () => { log(`Servidor de Autorização em http://localhost:${porta}`); resolve(s); });
  });
}

if (require.main === module) iniciar().catch((e) => { console.error(e.message); process.exit(1); });
module.exports = { iniciar };
```

Resultado esperado: Você aponta os dois eventos que contam como falha de autenticação para a regra de força bruta.

✔ EXEMPLO DE SUCESSO

```
node --check src\auth-server.js   (sem saída)
```

✘ EXEMPLO DE FALHA TÍPICA

```
ReferenceError: auditoria is not defined
```

Como corrigir: Falta const auditoria = require('./auditoria'); no topo do arquivo.

Passo 7 Instrumentar a API e o App Finanças

VS Code

A API registra `ACESSO_PERMITIDO`, `ACESSO_NEGADO` (401/403 com motivo), `OPERACAO_SENSIVEL` e `ALERTA_TOKEN_ADULTERADO` (CRÍTICO). O App Finanças repassa o seu `X-Request-Id` ao provedor e à API: uma ação do usuário vira um rastro único nos três serviços.

`src/api-financeira.js` (118 linhas)

src/api-financeira.js

```
// ============================================================
// api-financeira.js — SERVIDOR DE RECURSOS (porta 5000)
// Valida o access token (assinatura, expiração, emissor,
// audiência) e exige o ESCOPO certo para cada rota.
// Lab 2: registra acessos permitidos/negados, operações
// sensíveis e alerta tokens adulterados.
// ============================================================
const http = require('node:http');
const cfg = require('./config');
const jwt = require('./jwt');
const { lerCorpo, json } = require('./util');
const auditoria = require('./auditoria');

const audit = (req, tipo, extra = {}) =>
  auditoria.registrar({ origem: 'api', tipo, req, rota: new URL(req.url, 'http://local').pathname,
...extra });

const log = (...a) => { if (!process.env.OAUTH_QUIET) console.log('[api]', ...a); };

const contas = {
  u1: {
    saldo: 2350.75,
    transacoes: [
      { id: 1, data: '2026-10-01', descricao: 'Salário', valor: 3200.0 },
      { id: 2, data: '2026-10-05', descricao: 'Aluguel', valor: -1100.0 },
      { id: 3, data: '2026-10-12', descricao: 'Supermercado', valor: -412.35 },
      { id: 4, data: '2026-10-20', descricao: 'Conta de energia', valor: -186.9 },
      { id: 5, data: '2026-10-28', descricao: 'Pix recebido', valor: 850.0 },
    ],
  },
};

// Middleware de autorização: devolve as claims ou responde 401/403
```

```
function autorizar(req, res, escopoExigido) {
  const auth = req.headers.authorization || '';
  if (!auth.startsWith('Bearer ')) {
    audit(req, 'ACESSO_NEGADO', { nivel: 'AVISO', status: 401, motivo: 'sem token', escopo_exigido:
escopoExigido });
    json(res, 401, { error: 'invalid_token', error_description: 'Envie o cabeçalho Authorization:
Bearer <access_token>.' }, { 'WWW-Authenticate': 'Bearer realm="api-financeira"' });
    return null;
  }
  let claims;
  try {
    claims = jwt.verificar(auth.slice(7), cfg.SEGREDO_ASSINATURA);
  } catch (e) {
    const claims = jwt.decodificar(auth.slice(7)) || {};
    if (e.message.startsWith('assinatura')) {
      // assinatura inválida = alguém fabricou ou alterou o token: evento CRÍTICO
      audit(req, 'ALERTA_TOKEN_ADULTERADO', { nivel: 'CRITICO', status: 401, sub_alegado:
claims.sub, scope_alegado: claims.scope, motivo: e.message });
    } else {
      audit(req, 'ACESSO_NEGADO', { nivel: 'AVISO', status: 401, motivo: e.message, sub:
claims.sub, client_id: claims.client_id, jti: claims.jti });
    }
    json(res, 401, { error: 'invalid_token', error_description: e.message }, { 'WWW-Authenticate':
'Bearer realm="api-financeira", error="invalid_token"' });
    return null;
  }
  if (claims.iss !== cfg.EMISSOR || claims.aud !== cfg.AUDIENCIA_API) {
    audit(req, 'ACESSO_NEGADO', { nivel: 'AVISO', status: 401, motivo: 'iss/aud inválidos', sub:
claims.sub, jti: claims.jti });
    json(res, 401, { error: 'invalid_token', error_description: 'Emissor (iss) ou audiência (aud)
inválidos.' }, { 'WWW-Authenticate': 'Bearer error="invalid_token"' });
    return null;
  }
  const escopos = String(claims.scope || '').split(' ');
  if (!escopos.includes(escopoExigido)) {
    audit(req, 'ACESSO_NEGADO', { nivel: 'AVISO', status: 403, motivo: 'insufficient_scope',
escopo_exigido: escopoExigido, sub: claims.sub, client_id: claims.client_id, jti: claims.jti });
    json(res, 403, { error: 'insufficient_scope', error_description: `Este recurso exige o escopo
"${escopoExigido}".`, escopos_do_token: escopos },
      { 'WWW-Authenticate': `Bearer error="insufficient_scope", scope="${escopoExigido}"` });
    return null;
  }
  audit(req, 'ACESSO_PERMITIDO', { escopo: escopoExigido, sub: claims.sub, client_id:
claims.client_id, jti: claims.jti });
```

```
  return claims;
}

function criarServidor() {
  return http.createServer(auditoria.envolver('api', async (req, res) => {
    const url = new URL(req.url, `http://localhost:${cfg.PORTAS.api}`);
    log(req.method, url.pathname);
    try {
      if (req.method === 'GET' && url.pathname === '/') {
        return json(res, 200, { api: 'API Financeira (didática)', rotas: { 'GET /saldo':
'saldo:read', 'GET /transacoes': 'transacoes:read', 'POST /transferencias': 'transacoes:write',
'GET /relatorios/resumo': 'relatorios:read' } });
      }
      if (req.method === 'GET' && url.pathname === '/saldo') {
        const c = autorizar(req, res, 'saldo:read'); if (!c) return;
        return json(res, 200, { usuario: c.sub, saldo: contas[c.sub]?.saldo ?? 0, moeda: 'BRL' });
      }
      if (req.method === 'GET' && url.pathname === '/transacoes') {
        const c = autorizar(req, res, 'transacoes:read'); if (!c) return;
        return json(res, 200, { usuario: c.sub, transacoes: contas[c.sub]?.transacoes ?? [] });
      }
      if (req.method === 'POST' && url.pathname === '/transferencias') {
        const c = autorizar(req, res, 'transacoes:write'); if (!c) return;
        const corpo = await lerCorpo(req);
        audit(req, 'OPERACAO_SENSIVEL', { operacao: 'transferencia', sub: c.sub, client_id:
c.client_id, valor: Number(corpo.valor) || 0, destino: String(corpo.destino || '').slice(0, 40) });
        return json(res, 201, { status: 'transferência registrada', usuario: c.sub, ...corpo });
      }
      if (req.method === 'GET' && url.pathname === '/relatorios/resumo') {
        const c = autorizar(req, res, 'relatorios:read'); if (!c) return;
        const todas = Object.values(contas).flatMap((x) => x.transacoes);
        const entradas = todas.filter((t) => t.valor > 0).reduce((s, t) => s + t.valor, 0);
        const saidas = todas.filter((t) => t.valor < 0).reduce((s, t) => s + t.valor, 0);
        return json(res, 200, { solicitante: c.sub, periodo: '2026-10', entradas, saidas:
Math.abs(saidas), quantidade: todas.length });
      }
      json(res, 404, { error: 'not_found' });
    } catch (e) {
      console.error('[api] erro:', e);
      audit(req, 'ERRO_SERVIDOR', { nivel: 'ERRO', motivo: e.message });
      json(res, 500, { error: 'server_error' });
    }
  }));
}
```

```
function iniciar(porta = cfg.PORTAS.api) {
  return new Promise((resolve, reject) => {
    const s = criarServidor();
    s.once('error', reject);
    s.listen(porta, () => { log(`API Financeira em http://localhost:${porta}`); resolve(s); });
  });
}

if (require.main === module) iniciar().catch((e) => { console.error(e.message); process.exit(1);
});
module.exports = { iniciar };
```

`src/cliente-web.js` (182 linhas)

src/cliente-web.js

```
// ============================================================
// cliente-web.js — APLICAÇÃO CLIENTE "App Finanças" (porta 3000)
// Cliente confidencial: o back-end guarda client_secret, tokens e
// code_verifier; o navegador recebe apenas um cookie de sessão.
// Lab 2: propaga o X-Request-Id para provedor e API (rastreio ponta a ponta).
// ============================================================
const http = require('node:http');
const cfg = require('./config');
const jwt = require('./jwt');
const { lerCookies, html, redirecionar, aleatorio, sha256b64url, esc, pagina } = require('./util');
const auditoria = require('./auditoria');

const AUTH = cfg.EMISSOR;
const API = `http://localhost:${cfg.PORTAS.api}`;
const CLIENTE = cfg.CLIENTES.find((c) => c.client_id === 'app-financas-web');
const REDIRECT_URI = CLIENTE.redirect_uris[0];
const ESCOPOS_PEDIDOS = 'saldo:read transacoes:read'; // menor privilégio: NÃO pede
transacoes:write
const sessoes = new Map();
const log = (...a) => { if (!process.env.OAUTH_QUIET) console.log('[cliente]', ...a); };

function obterSessao(req, res) {
  let sid = lerCookies(req).sid;
  if (!sid || !sessoes.has(sid)) {
    sid = aleatorio(16);
    sessoes.set(sid, { log: [] });
    res.setHeader('Set-Cookie', `sid=${sid}; HttpOnly; SameSite=Lax; Path=/`);
  }
```

```
  return sessoes.get(sid);
}

async function pedirToken(params, reqId) {
  const basic =
Buffer.from(`${encodeURIComponent(CLIENTE.client_id)}:${encodeURIComponent(CLIENTE.client_secret)}`
).toString('base64');
  const r = await fetch(`${AUTH}/token`, {
    method: 'POST',
    headers: { 'Content-Type': 'application/x-www-form-urlencoded', Authorization: `Basic
${basic}`, 'X-Request-Id': reqId },
    body: new URLSearchParams(params),
  });
  return { status: r.status, corpo: await r.json() };
}

const resumir = (t) => (t ? `${t.slice(0, 16)}…` : undefined);
function guardarTokens(s, corpo) {
  s.refreshAnterior = s.tokens?.refresh_token;
  s.tokens = corpo;
}

function telaInicial(s) {
  const t = s.tokens;
  const claims = t ? jwt.decodificar(t.access_token) : null;
  const restante = claims ? claims.exp - Math.floor(Date.now() / 1000) : 0;
  const blocoLogin = `<p>Você ainda não autorizou o App Finanças a acessar sua conta.</p>
    <a class="btn" href="/login">Entrar com o Provedor OAuth</a>
    <a class="btn perigo" href="/login-redirect-invalido">Testar redirect_uri NÃO registrada</a>`;
  const blocoLogado = () => `<h2>Access token — claims (decodificado, sem validar)</h2>
    <pre>${esc(JSON.stringify(claims, null, 2))}</pre>
    <p>Expira em: <b>${restante > 0 ? `${restante} s` : 'EXPIRADO'}</b> · refresh token:
<code>${esc(resumir(t.refresh_token))}</code></p>
    <div>
      <a class="btn" href="/chamar/saldo">GET /saldo</a>
      <a class="btn" href="/chamar/transacoes">GET /transacoes</a>
      <form method="post" action="/transferir"><button class="perigo">POST /transferencias (escopo
não concedido)</button></form>
    </div><div>
      <form method="post" action="/renovar"><button class="sec">Renovar token (refresh)</button>
</form>
      ${s.refreshAnterior ? '<form method="post" action="/reusar-refresh"><button
class="perigo">Reutilizar refresh token ANTIGO</button></form>' : ''}
      <form method="post" action="/sair"><button class="sec">Sair (revogar)</button></form>
```

```
      <a class="btn sec" href="/">Atualizar</a>
    </div>`;
  const u = s.ultimo;
  return pagina('App Finanças — cliente OAuth', `
    <p class="tag">Aplicação cliente · App Finanças · porta ${cfg.PORTAS.cliente}</p>
    <h1>App Finanças</h1>
    ${t ? blocoLogado() : blocoLogin}
    ${u ? `<h2>Última operação: ${esc(u.titulo)} → HTTP ${u.status}</h2>
<pre>${esc(JSON.stringify(u.corpo, null, 2))}</pre>` : ''}
    ${s.log.length ? `<h2>Linha do tempo do fluxo</h2><ol>${s.log.map((l) => `<li>${esc(l)}
</li>`).join('')}</ol>` : ''}`);
}

function urlAutorizacao(s, redirectUri) {
  s.state = aleatorio(16); // anti-CSRF
  s.verifier = aleatorio(32); // PKCE: segredo que só o back-end do cliente conhece
  const challenge = sha256b64url(s.verifier);
  const u = new URL(`${AUTH}/authorize`);
  u.search = new URLSearchParams({ response_type: 'code', client_id: CLIENTE.client_id,
redirect_uri: redirectUri, scope: ESCOPOS_PEDIDOS, state: s.state, code_challenge: challenge,
code_challenge_method: 'S256' });
  s.log = [
    `Cliente gerou state=${s.state.slice(0, 8)}… e um code_verifier secreto; calculou
code_challenge=${challenge.slice(0, 10)}… (SHA-256).`,
    `Navegador redirecionado para ${AUTH}/authorize pedindo os escopos "${ESCOPOS_PEDIDOS}".`,
  ];
  return u.toString();
}

async function callback(s, url, res) {
  const q = url.searchParams;
  if (q.get('error')) {
    s.ultimo = { titulo: 'Autorização não concedida', status: 400, corpo: Object.fromEntries(q) };
    s.log.push(`Provedor devolveu erro: ${q.get('error')}.`);
    return redirecionar(res, '/');
  }
  if (!s.state || q.get('state') !== s.state) {
    auditoria.registrar({ origem: 'cliente', tipo: 'CALLBACK_STATE_INVALIDO', nivel: 'AVISO', req:
s.req, motivo: 'state ausente ou diferente (possível CSRF)' });
    return html(res, 400, pagina('Erro', '<h1>state inválido</h1><p class="erro">Possível ataque
CSRF: esta resposta não corresponde a um pedido iniciado nesta sessão.</p><a class="btn"
href="/">Voltar</a>'));
  }
  const code = q.get('code');
```

```
  s.log.push(`Callback recebeu code=${code.slice(0, 8)}… e o state conferiu.`);
  const r = await pedirToken({ grant_type: 'authorization_code', code, redirect_uri: REDIRECT_URI,
code_verifier: s.verifier }, s.reqId);
  delete s.state;
  delete s.verifier;
  if (r.status !== 200) {
    s.ultimo = { titulo: 'Troca do código falhou', ...r };
    return redirecionar(res, '/');
  }
  s.log.push('Back-end do cliente trocou code + code_verifier + client_secret por tokens em POST
/token (canal servidor-servidor).');
  guardarTokens(s, r.corpo);
  s.ultimo = { titulo: 'Tokens recebidos (valores abreviados)', status: 200, corpo: { ...r.corpo,
access_token: resumir(r.corpo.access_token), refresh_token: resumir(r.corpo.refresh_token) } };
  redirecionar(res, '/');
}

async function chamarApi(s, metodo, caminho, corpo) {
  const r = await fetch(`${API}${caminho}`, {
    method: metodo,
    headers: { Authorization: `Bearer ${s.tokens.access_token}`, 'Content-Type':
'application/json', 'X-Request-Id': s.reqId },
    body: corpo ? JSON.stringify(corpo) : undefined,
  });
  s.ultimo = { titulo: `${metodo} ${caminho}`, status: r.status, corpo: await r.json() };
  s.log.push(`Chamou ${metodo} ${caminho} com Authorization: Bearer → HTTP ${r.status}.`);
}

async function renovar(s, refresh, titulo) {
  const r = await pedirToken({ grant_type: 'refresh_token', refresh_token: refresh }, s.reqId);
  if (r.status === 200) {
    guardarTokens(s, r.corpo);
    s.ultimo = { titulo, status: 200, corpo: { ...r.corpo, access_token:
resumir(r.corpo.access_token), refresh_token: resumir(r.corpo.refresh_token) } };
    s.log.push('Refresh token trocado por um NOVO par de tokens (rotação: o refresh antigo foi
invalidado).');
  } else {
    s.ultimo = { titulo, ...r };
    s.log.push(`Renovação recusada: ${r.corpo.error_description}`);
    if (String(r.corpo.error_description).includes('REUSO')) delete s.tokens; // sessão
comprometida
  }
}
```

```
function criarServidor() {
  return http.createServer(auditoria.envolver('cliente', async (req, res) => {
    const url = new URL(req.url, `http://localhost:${cfg.PORTAS.cliente}`);
    const s = obterSessao(req, res);
    s.reqId = req.idReq; // o mesmo ID segue nas chamadas ao provedor e à API
    s.req = req;
    log(req.method, url.pathname);
    try {
      if (req.method === 'GET' && url.pathname === '/') return html(res, 200, telaInicial(s));
      if (req.method === 'GET' && url.pathname === '/login') return redirecionar(res,
urlAutorizacao(s, REDIRECT_URI));
      if (req.method === 'GET' && url.pathname === '/login-redirect-invalido') return
redirecionar(res, urlAutorizacao(s, 'http://site-malicioso.exemplo/callback'));
      if (req.method === 'GET' && url.pathname === '/callback') return await callback(s, url, res);
      if (!s.tokens) return redirecionar(res, '/'); // rotas abaixo exigem login
      if (req.method === 'GET' && url.pathname === '/chamar/saldo') await chamarApi(s, 'GET',
'/saldo');
      else if (req.method === 'GET' && url.pathname === '/chamar/transacoes') await chamarApi(s,
'GET', '/transacoes');
      else if (req.method === 'POST' && url.pathname === '/transferir') await chamarApi(s, 'POST',
'/transferencias', { destino: 'Conta 9999-0', valor: 100 });
      else if (req.method === 'POST' && url.pathname === '/renovar') await renovar(s,
s.tokens.refresh_token, 'Renovação com refresh token atual');
      else if (req.method === 'POST' && url.pathname === '/reusar-refresh' && s.refreshAnterior)
await renovar(s, s.refreshAnterior, 'Tentativa de REUSO do refresh token antigo');
      else if (req.method === 'POST' && url.pathname === '/sair') {
        const basic =
Buffer.from(`${CLIENTE.client_id}:${CLIENTE.client_secret}`).toString('base64');
        await fetch(`${AUTH}/revoke`, { method: 'POST', headers: { Authorization: `Basic ${basic}`,
'Content-Type': 'application/x-www-form-urlencoded', 'X-Request-Id': s.reqId }, body: new
URLSearchParams({ token: s.tokens.refresh_token }) });
        delete s.tokens;
        delete s.refreshAnterior;
        s.ultimo = { titulo: 'Logout', status: 200, corpo: { mensagem: 'Refresh token revogado no
provedor e sessão local encerrada.' } };
        s.log = [];
      }
      redirecionar(res, '/'); // padrão Post/Redirect/Get
    } catch (e) {
      console.error('[cliente] erro:', e);
      html(res, 502, pagina('Erro', `<h1>Falha ao falar com o provedor/API</h1>
<pre>${esc(e.message)}</pre><p>Os três servidores estão rodando? Use <code>npm start</code>.
</p>`));
    }
```

```
  }));
}

function iniciar(porta = cfg.PORTAS.cliente) {
  return new Promise((resolve, reject) => {
    const s = criarServidor();
    s.once('error', reject);
    s.listen(porta, () => { log(`App Finanças em http://localhost:${porta}`); resolve(s); });
  });
}

if (require.main === module) iniciar().catch((e) => { console.error(e.message); process.exit(1);
});
module.exports = { iniciar };
```

Resultado esperado: Você explica a diferença entre um 401 por token expirado e um por assinatura inválida.

✔ EXEMPLO DE SUCESSO

```
node --check src\api-financeira.js; node --check src\cliente-web.js
```

✘ EXEMPLO DE FALHA TÍPICA

```
SyntaxError: missing ) after argument list   (faltou o ')' de
auditoria.envolver(...))
```

Como corrigir: O createServer precisa terminar em '}));' — confira o fechamento.

Passo 8 Criar simulador, analisador e scripts

VS Code

`simular-trafego.js` gera uso normal e cinco ataques (força bruta de senha, token adulterado, escalonamento de escopo, reuso de refresh e varredura de `client_secret`) usando IPs de faixas de documentação. `analisar-logs.js` verifica a integridade, calcula indicadores, correlaciona por IP e `req_id` e gera `relatorio/relatorio-auditoria.html`.

`src/iniciar.js` (29 linhas)

src/iniciar.js

```
// ============================================================
// iniciar.js — Sobe os três papéis do OAuth no mesmo processo
// ============================================================
const cfg = require('./config');
const auth = require('./auth-server');
const api = require('./api-financeira');
```

```
const cliente = require('./cliente-web');

(async () => {
  await auth.iniciar();
  await api.iniciar();
  await cliente.iniciar();
  console.log(`
============================================================
 Laboratório OAuth em execução
  Servidor de Autorização : ${cfg.EMISSOR}
  API Financeira (recursos): http://localhost:${cfg.PORTAS.api}
  App Finanças (cliente)   : http://localhost:${cfg.PORTAS.cliente}   <-- abra no navegador
  Usuário demo: aluno / senha123   | access token: ${cfg.ACCESS_TOKEN_TTL}s
  Logs de auditoria        : ${cfg.AUDITORIA.LOG_DIR}  (auditoria.log, alertas.log)
  Força bruta: ${cfg.AUDITORIA.LIMITE_FALHAS} falhas/${cfg.AUDITORIA.JANELA_S}s => bloqueio de
${cfg.AUDITORIA.BLOQUEIO_S}s
  Pressione Ctrl+C para encerrar.
============================================================`);
})().catch((e) => {
  console.error(`Falha ao iniciar: ${e.message}`);
  if (e.code === 'EADDRINUSE') console.error('Porta ocupada. Feche o outro servidor (Ctrl+C) ou
defina AUTH_PORT/API_PORT/CLIENT_PORT.');
  process.exit(1);
});
```

`src/simular-trafego.js` (96 linhas)

src/simular-trafego.js

```
// ============================================================
// simular-trafego.js — Gera tráfego NORMAL + ATAQUES para análise
// Uso (com "npm start" rodando em outro terminal): npm run simular
// IPs simulados via X-Forwarded-For (faixas reservadas para
// documentação: 10.0.0.0/8 interna, 203.0.113.0/24 e 198.51.100.0/24)
// ============================================================
const cfg = require('./config');
const jwt = require('./jwt');
const { aleatorio, sha256b64url } = require('./util');

const AUTH = cfg.EMISSOR;
const API = `http://localhost:${cfg.PORTAS.api}`;
const WEB = cfg.CLIENTES.find((c) => c.client_id === 'app-financas-web');
const SVC = cfg.CLIENTES.find((c) => c.client_id === 'servico-relatorios');
const IP_APP = '10.0.0.5'; // servidor do App Finanças (canal back-end)
const basic = (c) => `Basic
```

```
${Buffer.from(`${c.client_id}:${c.client_secret}`).toString('base64')}`;
const cont = { normal: 0, ataque: 0, status: {} };

async function req(url, { ip, metodo = 'GET', corpo, headers = {}, ataque = false } = {}) {
  const r = await fetch(url, { method: metodo, body: corpo, redirect: 'manual', headers: { 'X-
Forwarded-For': ip, 'User-Agent': ataque ? 'python-requests/2.31' : 'Mozilla/5.0 (Lab)', ...headers
} });
  cont[ataque ? 'ataque' : 'normal']++;
  cont.status[r.status] = (cont.status[r.status] || 0) + 1;
  return r;
}

async function login(ipUsuario, senha = 'senha123', ataque = false, usuario = 'aluno') {
  const verifier = aleatorio(32);
  const corpo = new URLSearchParams({
    response_type: 'code', client_id: WEB.client_id, redirect_uri: WEB.redirect_uris[0], scope:
'saldo:read transacoes:read',
    state: aleatorio(8), code_challenge: sha256b64url(verifier), code_challenge_method: 'S256',
login: usuario, senha, decisao: 'permitir',
  });
  const r = await req(`${AUTH}/authorize`, { ip: ipUsuario, metodo: 'POST', corpo, ataque });
  const loc = r.headers.get('location');
  return { status: r.status, code: loc ? new URL(loc).searchParams.get('code') : null, verifier };
}

async function token(params, cliente = WEB, ip = IP_APP, ataque = false) {
  const r = await req(`${AUTH}/token`, { ip, metodo: 'POST', corpo: new URLSearchParams(params),
headers: { Authorization: basic(cliente) }, ataque });
  return { status: r.status, corpo: await r.json() };
}

const api = (caminho, tk, ip = IP_APP, metodo = 'GET', ataque = false) =>
  req(`${API}${caminho}`, { ip, metodo, headers: { Authorization: `Bearer ${tk}`, 'Content-Type':
'application/json' }, corpo: metodo === 'POST' ? '{"destino":"Conta 9999-0","valor":5000}' :
undefined, ataque });

async function sessaoNormal(ipUsuario) {
  const l = await login(ipUsuario);
  if (!l.code) throw new Error(`login normal falhou (HTTP ${l.status}) — o IP ${ipUsuario} está
bloqueado?`);
  const t = await token({ grant_type: 'authorization_code', code: l.code, redirect_uri:
WEB.redirect_uris[0], code_verifier: l.verifier });
  for (const c of ['/saldo', '/transacoes', '/saldo']) await api(c, t.corpo.access_token);
  return t.corpo;
```

```
}

(async () => {
  console.log('Simulando tráfego... (servidores precisam estar no ar: npm start)\n');

  console.log('1) Uso normal: 3 usuários fazem login e consultam saldo/extrato');
  const sessoes = [];
  for (const ip of ['10.0.0.11', '10.0.0.12', '10.0.0.13']) sessoes.push(await sessaoNormal(ip));
  await token({ grant_type: 'refresh_token', refresh_token: sessoes[0].refresh_token }); //
renovação legítima

  console.log('2) Serviço de Relatórios (Client Credentials) gera o relatório do mês');
  const cc = await token({ grant_type: 'client_credentials', scope: 'relatorios:read' }, SVC,
'10.0.0.20');
  await api('/relatorios/resumo', cc.corpo.access_token, '10.0.0.20');

  console.log('3) ATAQUE: força bruta de senha a partir de 203.0.113.66');
  const tentativas = [];
  const chutes = [['admin', 'admin'], ['root', '123456'], ['aluno', 'aluno123'], ['aluno',
'qwerty'], ['aluno', 'senha1234'], ['aluno', 'Senha123'], ['aluno', 'senha123'], ['aluno',
'senha123']];
  for (const [usuario, senha] of chutes) tentativas.push((await login('203.0.113.66', senha, true,
usuario)).status);
  console.log(`   status recebidos: ${tentativas.join(', ')}  (429 = IP bloqueado; nem a senha
certa passa)`);

  console.log('4) ATAQUE: token adulterado (escopo alterado) a partir de 198.51.100.23');
  const [h, , s] = sessoes[1].access_token.split('.');
  const falso = Buffer.from(JSON.stringify({ ...jwt.decodificar(sessoes[1].access_token), scope:
'saldo:read transacoes:read transacoes:write' })).toString('base64url');
  for (let i = 0; i < 3; i++) await api('/transferencias', `${h}.${falso}.${s}`, '198.51.100.23',
'POST', true);

  console.log('5) ATAQUE: escalonamento — token legítimo tentando transferir sem escopo');
  for (let i = 0; i < 4; i++) await api('/transferencias', sessoes[2].access_token, '10.0.0.13',
'POST', true);

  console.log('6) ATAQUE: reuso de refresh token roubado (já trocado no passo 1)');
  await token({ grant_type: 'refresh_token', refresh_token: sessoes[0].refresh_token }, WEB,
'198.51.100.23', true);

  console.log('7) ATAQUE: varredura de client_secret a partir de 203.0.113.99');
  for (const seg of ['secret', 'segredo', '123', 'admin', 'teste', 'segredo-app']) {
    await token({ grant_type: 'client_credentials' }, { client_id: 'app-financas-web',
```

```
client_secret: seg }, '203.0.113.99', true);
  }

  console.log('8) Acessos sem token (robôs varrendo a API)');
  for (const c of ['/saldo', '/transacoes', '/admin', '/.env']) await req(`${API}${c}`, { ip:
'198.51.100.77', ataque: true });

  console.log(`\nConcluído: ${cont.normal} requisições normais e ${cont.ataque} de ataque. Status:
${JSON.stringify(cont.status)}`);
  console.log('Próximo passo: npm run analisar');
})().catch((e) => {
  console.error(`Erro: ${e.message}. O laboratório está rodando? Execute "npm start" em outro
terminal.`);
  process.exit(1);
});
```

`src/analisar-logs.js` (174 linhas)

src/analisar-logs.js

```
// ============================================================
// analisar-logs.js — ANÁLISE dos logs de auditoria
// Verifica a integridade, calcula indicadores (KPIs), correlaciona
// eventos por IP / usuário / req_id e gera relatório HTML + JSON.
// Uso:  npm run analisar
//       npm run analisar -- --decifrar        (mostra logins digitados errado)
//       npm run analisar -- --req=<req_id>    (rastreia uma requisição ponta a ponta)
//       npm run analisar -- --dir=<pasta>     (outra pasta de logs)
// Código de saída: 0 = ok | 2 = integridade violada
// ============================================================
const fs = require('node:fs');
const path = require('node:path');
const cfg = require('./config');
const auditoria = require('./auditoria');

const contar = (lista, chave) => lista.reduce((m, x) => { const k = chave(x); if (k !== undefined)
m[k] = (m[k] || 0) + 1; return m; }, {});
const top = (obj, n = 5) => Object.entries(obj).sort((a, b) => b[1] - a[1]).slice(0, n);
const pct = (a, b) => (b ? Math.round((a / b) * 1000) / 10 : 0);
function p95(valores) {
  if (!valores.length) return 0;
  const v = [...valores].sort((a, b) => a - b);
  return v[Math.min(v.length - 1, Math.ceil(v.length * 0.95) - 1)];
}
```

```
function analisar(dir = cfg.AUDITORIA.LOG_DIR) {
  const integridade = auditoria.verificarIntegridade(dir);
  const ev = auditoria.lerEventos(dir);
  const http = ev.filter((e) => e.tipo === 'HTTP_REQUISICAO');
  const seguranca = ev.filter((e) => e.tipo !== 'HTTP_REQUISICAO');
  const criticos = ev.filter((e) => e.nivel === 'CRITICO');
  const loginsOk = ev.filter((e) => e.tipo === 'LOGIN_SUCESSO').length;
  const loginsFalha = ev.filter((e) => e.tipo === 'LOGIN_FALHA').length;
  const apiHttp = http.filter((e) => e.origem === 'api');
  const errosHttp = http.filter((e) => e.status >= 400).length;

  const porOrigem = {};
  for (const o of ['auth', 'api', 'cliente']) {
    const l = http.filter((e) => e.origem === o);
    porOrigem[o] = { total: l.length, erros_4xx: l.filter((e) => e.status >= 400 && e.status <
500).length, erros_5xx: l.filter((e) => e.status >= 500).length, p95_ms: p95(l.map((e) =>
e.duracao_ms)) };
  }

  // Correlação por IP: eventos de segurança com nível AVISO/CRITICO (exclui log de acesso)
  const suspeitos = seguranca.filter((e) => ['AVISO', 'CRITICO'].includes(e.nivel) && e.ip);
  const suspeitosPorIp = top(contar(suspeitos, (e) => e.ip), 8).map(([ip, n]) => ({
    ip, eventos: n, tipos: contar(suspeitos.filter((e) => e.ip === ip), (e) => e.tipo),
    bloqueado: ev.some((e) => e.tipo === 'ALERTA_FORCA_BRUTA' && e.ip === ip),
  }));

  // Linha do tempo por minuto
  const linhaDoTempo = Object.entries(ev.reduce((m, e) => {
    const k = e.ts.slice(0, 16);
    m[k] = m[k] || { total: 0, avisos: 0, criticos: 0 };
    m[k].total++;
    if (e.nivel === 'AVISO') m[k].avisos++;
    if (e.nivel === 'CRITICO') m[k].criticos++;
    return m;
  }, {})).map(([minuto, v]) => ({ minuto, ...v }));

  const taxaFalhaLogin = pct(loginsFalha, loginsOk + loginsFalha);
  const taxaErroHttp = pct(errosHttp, http.length);
  const p95Api = p95(apiHttp.map((e) => e.duracao_ms));
  const ipsBloqueados = [...new Set(ev.filter((e) => e.tipo === 'ALERTA_FORCA_BRUTA').map((e) =>
e.ip))];
  const kpis = [
    { nome: 'Integridade dos logs', valor: integridade.ok ? 'íntegros' :
`${integridade.erros.length} violação(ões)`, meta: 'íntegros', ok: integridade.ok },
```

```
    { nome: 'Alertas críticos', valor: criticos.length, meta: '0', ok: criticos.length === 0 },
    { nome: 'IPs bloqueados (força bruta)', valor: ipsBloqueados.length, meta: '0', ok:
ipsBloqueados.length === 0 },
    { nome: 'Taxa de falha de login', valor: `${taxaFalhaLogin}%`, meta: '< 20%', ok:
taxaFalhaLogin < 20 },
    { nome: 'Taxa de erro HTTP (4xx+5xx)', valor: `${taxaErroHttp}%`, meta: '< 10%', ok:
taxaErroHttp < 10 },
    { nome: 'Latência p95 da API', valor: `${p95Api} ms`, meta: '< 200 ms', ok: p95Api < 200 },
  ];

  const recomendacoes = [];
  if (!integridade.ok) recomendacoes.push('Logs adulterados ou corrompidos: preservar cópia, abrir
incidente e investigar acesso ao servidor de logs.');
  for (const s of suspeitosPorIp.filter((x) => x.bloqueado)) recomendacoes.push(`IP ${s.ip}: força
bruta (${s.eventos} eventos suspeitos) — manter bloqueio, avaliar bloqueio permanente no
firewall/WAF.`);
  if (ev.some((e) => e.tipo === 'ALERTA_TOKEN_ADULTERADO')) recomendacoes.push('Tokens com
assinatura inválida: alguém está forjando tokens — verificar exposição do segredo de assinatura e
migrar para chaves assimétricas (RS256/ES256).');
  if (ev.some((e) => e.tipo === 'REFRESH_REUSO')) recomendacoes.push('Reuso de refresh token:
sessão possivelmente roubada — avisar o usuário, revisar dispositivos e forçar novo login.');
  const escal = ev.filter((e) => e.tipo === 'ACESSO_NEGADO' && e.status === 403);
  if (escal.length >= 3) recomendacoes.push(`${escal.length} tentativas de acesso fora do escopo
(403): revisar o cliente ${[...new Set(escal.map((e) => e.client_id))].join(', ')} e criar alerta
para picos de 403.`);
  if (!recomendacoes.length) recomendacoes.push('Nenhuma anomalia relevante no período. Manter
revisão periódica.');

  return {
    gerado_em: new Date().toISOString(), pasta: dir,
    periodo: { inicio: ev[0]?.ts, fim: ev[ev.length - 1]?.ts },
    integridade, total_eventos: ev.length,
    por_tipo: contar(ev, (e) => e.tipo), por_nivel: contar(ev, (e) => e.nivel),
    http: { total: http.length, taxa_erro_pct: taxaErroHttp, por_origem: porOrigem, por_status:
contar(http, (e) => e.status) },
    autenticacao: { logins_sucesso: loginsOk, logins_falha: loginsFalha, taxa_falha_pct:
taxaFalhaLogin, falhas_cliente: ev.filter((e) => e.tipo === 'CLIENTE_AUTH_FALHA').length },
    suspeitos_por_ip: suspeitosPorIp, ips_bloqueados: ipsBloqueados,
    alertas: criticos.map((e) => ({ ts: e.ts, tipo: e.tipo, ip: e.ip, req_id: e.req_id, detalhe:
e.acao || e.motivo || '' })),
    negados_por_motivo: contar(ev.filter((e) => e.tipo === 'ACESSO_NEGADO'), (e) => `${e.status}
${e.motivo}`),
    linha_do_tempo: linhaDoTempo, kpis, recomendacoes,
  };
```

```
}

// ------------------------------------------------------------- relatório HTML
function htmlRelatorio(r) {
  const esc = (s) => String(s ?? '').replace(/[&<>"]/g, (c) => ({ '&': '&amp;', '<': '&lt;', '>': '&gt;', '"': '&quot;' }[c]));
  const maxIp = Math.max(1, ...r.suspeitos_por_ip.map((s) => s.eventos));
  const maxMin = Math.max(1, ...r.linha_do_tempo.map((m) => m.total));
  return `<!doctype html><html lang="pt-BR"><head><meta charset="utf-8"><meta name="viewport" content="width=device-width,initial-scale=1">
<title>Relatório de auditoria</title><style>
body{margin:0;background:#0e1116;color:#e6edf3;font:15px/1.5 "Segoe UI",system-ui,sans-serif}main{max-width:1000px;margin:0 auto;padding:20px 16px 48px}
h1{margin:.2rem 0}h2{font-size:1.1rem;margin:1.6rem 0 .6rem}.sub{color:#9aa6b2}
.kpis{display:grid;grid-template-columns:repeat(auto-fit,minmax(170px,1fr));gap:10px}.kpi{background:#161b22;border:1px solid #2a313c;border-radius:10px;padding:12px}
.kpi b{display:block;font-size:1.4rem;font-variant-numeric:tabular-nums}.kpi small{color:#9aa6b2}.kpi.ok{border-left:4px solid #3fb68b}.kpi.ruim{border-left:4px solid #f85149}
table{width:100%;border-collapse:collapse;font-size:.88rem}th,td{text-align:left;padding:.4rem;border-bottom:1px solid #2a313c;vertical-align:top}th{color:#9aa6b2}
.barra{height:12px;background:#f0883e;border-radius:3px}.barra.c{background:#f85149}.tl{display:flex;align-items:flex-end;gap:3px;height:120px;border-bottom:1px solid #2a313c}
.tl div{flex:1;max-width:48px;background:#3fb68b;min-width:4px;border-radius:2px 2px 0 0}.tl div.c{background:#f85149}.tag{display:inline-block;background:#222a35;border-radius:5px;padding:0 .4rem;margin:1px;font-size:.8rem}
.box{background:#161b22;border:1px solid #2a313c;border-radius:10px;padding:12px 14px}.erro{color:#ff7b72}.okc{color:#3fb68b}code{font-family:Consolas,monospace}
</style></head><body><main>
<p class="sub">Laboratório 2 · Auditoria e logs da API Financeira</p><h1>Relatório de auditoria</h1>
<p class="sub">Período: ${esc(r.periodo.inicio)} → ${esc(r.periodo.fim)} · ${r.total_eventos} eventos · gerado em ${esc(r.gerado_em)}</p>
<h2>Indicadores</h2><div class="kpis">${r.kpis.map((k) => `<div class="kpi ${k.ok ? 'ok' : 'ruim'}"><small>${esc(k.nome)}</small><b>${esc(k.valor)}</b><small>meta: ${esc(k.meta)} · ${k.ok ? 'OK' : 'ATENÇÃO'}</small></div>`).join('')}</div>
<h2>Integridade da cadeia de logs</h2><div class="box">${r.integridade.ok ? `<span class="okc">✔ ${r.integridade.total} registros verificados em ${r.integridade.arquivos.length} arquivo(s); nenhum alterado.</span>` : `<span class="erro">✘ ${r.integridade.erros.length} problema(s):</span>
<ul>${r.integridade.erros.slice(0, 20).map((e) => `<li class="erro">${esc(e.arquivo)} linha ${e.linha}: ${esc(e.motivo)}</li>`).join('')}</ul>`}
${r.integridade.ancora && !r.integridade.ancora.completa ? '<p class="sub">Início da cadeia em arquivo posterior ao GENESIS: arquivos mais antigos foram expurgados pela política de retenção.
```

```
</p>' : ''}</div>
<h2>Alertas críticos (${r.alertas.length})</h2><table><tr><th>Horário</th><th>Tipo</th><th>IP</th>
<th>Detalhe</th><th>req_id</th></tr>
${r.alertas.map((a) => `<tr><td>${esc(a.ts)}</td><td>${esc(a.tipo)}</td><td>${esc(a.ip)}</td>
<td>${esc(a.detalhe)}</td><td><code>${esc(String(a.req_id || '').slice(0, 8))}</code></td>
</tr>`).join('') || '<tr><td colspan="5">Nenhum</td></tr>'}</table>
<h2>IPs com mais eventos suspeitos</h2><table><tr><th>IP</th><th style="width:35%">Eventos</th>
<th>Tipos</th></tr>
${r.suspeitos_por_ip.map((s) => `<tr><td>${esc(s.ip)}${s.bloqueado ? ' <span
class="tag">bloqueado</span>' : ''}</td><td><div class="barra ${s.bloqueado ? 'c' : ''}"
style="width:${(s.eventos / maxIp) * 100}%"></div>${s.eventos}</td>
<td>${Object.entries(s.tipos).map(([t, n]) => `<span class="tag">${esc(t)} ×${n}</span>`).join('')}
</td></tr>`).join('')}</table>
<h2>Eventos por minuto</h2><div class="tl">${r.linha_do_tempo.map((m) => `<div class="${m.criticos
? 'c' : ''}" style="height:${(m.total / maxMin) * 100}%" title="${esc(m.minuto)}: ${m.total}
eventos, ${m.criticos} críticos"></div>`).join('')}</div>
<p class="sub">Barras vermelhas = minutos com eventos críticos.</p>
<h2>Acessos negados por motivo</h2><table>${Object.entries(r.negados_por_motivo).map(([k, v]) =>
`<tr><td>${esc(k)}</td><td>${v}</td></tr>`).join('') || '<tr><td>Nenhum</td></tr>'}</table>
<h2>Tráfego HTTP por serviço</h2><table><tr><th>Serviço</th><th>Requisições</th><th>4xx</th>
<th>5xx</th><th>p95</th></tr>
${Object.entries(r.http.por_origem).map(([o, v]) => `<tr><td>${o}</td><td>${v.total}</td>
<td>${v.erros_4xx}</td><td>${v.erros_5xx}</td><td>${v.p95_ms} ms</td></tr>`).join('')}</table>
<h2>Recomendações</h2><ul>${r.recomendacoes.map((x) => `<li>${esc(x)}</li>`).join('')}</ul>
</main></body></html>`;
}

// ------------------------------------------------------------------ CLI
if (require.main === module) {
  const args = Object.fromEntries(process.argv.slice(2).map((a) => { const [k, v] = a.replace(/^-
-/, '').split('='); return [k, v ?? true]; }));
  const dir = args.dir || cfg.AUDITORIA.LOG_DIR;
  if (!auditoria.listarArquivos(dir).length) {
    console.error(`Nenhum log em ${dir}. Rode "npm start" e gere tráfego ("npm run simular")
antes.`);
    process.exit(1);
  }
  if (args.req) {
    const lista = auditoria.lerEventos(dir).filter((e) => e.req_id === args.req);
    console.log(`Rastreio do req_id ${args.req} (${lista.length} eventos):`);
    for (const e of lista) console.log(`  ${e.ts} [${e.origem}] ${e.tipo} ${e.metodo || ''}
${e.rota || ''} ${e.status || ''} ${e.motivo || ''}`);
    process.exit(0);
  }
```

```
  const r = analisar(dir);
  const L = (s = '') => console.log(s);
  L('==================== ANÁLISE DOS LOGS DE AUDITORIA ====================');
  L(`Pasta: ${dir}`);
  L(`Período: ${r.periodo.inicio} -> ${r.periodo.fim} | ${r.total_eventos} eventos`);
  L(`Integridade: ${r.integridade.ok ? 'OK' : 'VIOLADA'} (${r.integridade.total} registros,
${r.integridade.arquivos.length} arquivo(s))`);
  for (const e of r.integridade.erros.slice(0, 10)) L(`   ! ${e.arquivo} linha ${e.linha}:
${e.motivo}`);
  L('\nIndicadores:');
  for (const k of r.kpis) L(`  [${k.ok ? 'OK' : 'ATENCAO'}] ${k.nome}: ${k.valor} (meta
${k.meta})`);
  L(`\nAlertas críticos: ${r.alertas.length}`);
  for (const a of r.alertas) L(`  ${a.ts} ${a.tipo} ip=${a.ip} ${a.detalhe}`);
  L('\nIPs com mais eventos suspeitos:');
  for (const s of r.suspeitos_por_ip) L(`  ${s.ip.padEnd(16)} ${String(s.eventos).padStart(3)}
${s.bloqueado ? '[BLOQUEADO] ' : ''}${Object.keys(s.tipos).join(', ')}`);
  if (args.decifrar) {
    L('\nLogins digitados nas falhas (decifrados — acesso restrito à equipe de segurança):');
    for (const e of auditoria.lerEventos(dir).filter((x) => x.tipo === 'LOGIN_FALHA')) {
      let login; try { login = auditoria.decifrar(e.login_cifrado); } catch { login = '(chave
incorreta)'; }
      L(`  ${e.ts} ip=${e.ip} login="${login}"`);
    }
  }
  L('\nRecomendações:');
  for (const x of r.recomendacoes) L(`  - ${x}`);
  const pasta = path.join(__dirname, '..', 'relatorio');
  fs.mkdirSync(pasta, { recursive: true });
  fs.writeFileSync(path.join(pasta, 'indicadores.json'), JSON.stringify(r, null, 2));
  fs.writeFileSync(path.join(pasta, 'relatorio-auditoria.html'), htmlRelatorio(r));
  L(`\nRelatório: ${path.join(pasta, 'relatorio-auditoria.html')}`);
  process.exit(r.integridade.ok ? 0 : 2);
}


module.exports = { analisar, htmlRelatorio };
```

`package.json` (20 linhas)

package.json

```
{
  "name": "atividade-oauth-api",
  "version": "2.0.0",
  "description": "Laboratório OAuth 2.0 + Auditoria e Logs — API Financeira (Programação Back-End,
```

```
aulas 1 e 3)",
  "main": "src/iniciar.js",
  "private": true,
  "scripts": {
    "start": "node src/iniciar.js",
    "test": "node testes/teste-fluxo.js && node testes/teste-auditoria.js",
    "test:auditoria": "node testes/teste-auditoria.js",
    "cc": "node src/client-credentials.js",
    "simular": "node src/simular-trafego.js",
    "analisar": "node src/analisar-logs.js"
  },
  "engines": {
    "node": ">=20"
  },
  "license": "UNLICENSED"
}
```

`README.md` (24 linhas)

README.md

```
# Laboratório OAuth 2.0 + Auditoria e Logs — API Financeira

Programação Back-End (2ª série, 4º bimestre, semana 25): **aula 1** (OAuth) e **aula 3** (auditoria
e logs). Sem dependências externas: apenas Node.js 20+.

| Papel | Arquivo | Porta |
|---|---|---|
| Servidor de Autorização | `src/auth-server.js` | 4000 |
| Servidor de Recursos (API) | `src/api-financeira.js` | 5000 |
| Cliente (App Finanças) | `src/cliente-web.js` | 3000 |
| Auditoria (logs, alertas, integridade) | `src/auditoria.js` | — |

```powershell
npm test               # 25 testes OAuth + 27 testes de auditoria
npm start              # sobe os três servidores -> http://localhost:3000 (aluno / senha123)
npm run simular        # (outro terminal) tráfego normal + ataques
npm run analisar       # KPIs, alertas, integridade -> relatorio\relatorio-auditoria.html
node src\analisar-logs.js --decifrar        # logins digitados nas falhas (restrito)
node src\analisar-logs.js --req=<req_id>    # rastreia uma requisição ponta a ponta
$env:LOG_MAX_BYTES=8000; npm start          # força rotação rápida dos logs
```
```

```
Logs em `logs/` (fora do Git): `auditoria.log` (+ rotacionados) e `alertas.log`.
Planos do grupo: `docs/plano-oauth.md` (aula 1) e `docs/plano-auditoria.md` (aula 3).
```

Resultado esperado: `package.json` com os scripts start, test, simular e analisar.

✔ EXEMPLO DE SUCESSO

```
npm run    (lista: start, test, test:auditoria, cc, simular, analisar)
```

✘ EXEMPLO DE FALHA TÍPICA

```
npm ERR! Missing script: "simular"
```

Como corrigir: Substitua o package.json inteiro pelo desta seção.

Passo 9 Rodar os testes automáticos

VS Code + PowerShell

`teste-auditoria.js` verifica 27 regras (correlação, eventos, alertas, bloqueio, ausência de segredos, integridade, rotação e análise) em uma pasta temporária. `teste-fluxo.js` recebeu só uma linha para também usar pasta temporária.

VS Code + PowerShell — comandos

```
npm test
```

`testes/teste-auditoria.js` (141 linhas)

testes/teste-auditoria.js

```
// ============================================================
// teste-auditoria.js — Testes automáticos do Laboratório 2
// (npm run test:auditoria) — logs, alertas, integridade, rotação
// ============================================================
const fs = require('node:fs');
const os = require('node:os');
const path = require('node:path');

const PASTA = fs.mkdtempSync(path.join(os.tmpdir(), 'auditoria-teste-'));
Object.assign(process.env, {
  AUTH_PORT: '24000', API_PORT: '25000', CLIENT_PORT: '23000', OAUTH_QUIET: '1',
  LOG_DIR: PASTA, LOG_MAX_BYTES: '20000', LOG_RETENCAO_ARQUIVOS: '50', LIMITE_FALHAS: '5',
JANELA_S: '60', BLOQUEIO_S: '300',
});

const cfg = require('../src/config');
const jwt = require('../src/jwt');
```

```
const auditoria = require('../src/auditoria');
const { analisar } = require('../src/analisar-logs');
const { sha256b64url, aleatorio } = require('../src/util');

const AUTH = cfg.EMISSOR;
const API = `http://localhost:${cfg.PORTAS.api}`;
const WEB = cfg.CLIENTES[0];
const basic = (c) => `Basic
${Buffer.from(`${c.client_id}:${c.client_secret}`).toString('base64')}`;
const eventos = (tipo) => auditoria.memoria.filter((e) => e.tipo === tipo);

let ok = 0;
let falhas = 0;
function verificar(descricao, condicao, detalhe = '') {
  if (condicao) { ok++; console.log(`  [OK]    ${descricao}`); }
  else { falhas++; console.log(`  [FALHA] ${descricao}${detalhe ? ` -> ${detalhe}` : ''}`); }
}

async function login(ip, senha = 'senha123') {
  const v = aleatorio(32);
  const r = await fetch(`${AUTH}/authorize`, {
    method: 'POST', redirect: 'manual', headers: { 'X-Forwarded-For': ip },
    body: new URLSearchParams({ response_type: 'code', client_id: WEB.client_id, redirect_uri:
WEB.redirect_uris[0], scope: 'saldo:read transacoes:read', state: 'st', code_challenge:
sha256b64url(v), code_challenge_method: 'S256', login: 'aluno', senha, decisao: 'permitir' }),
  });
  const loc = r.headers.get('location');
  return { status: r.status, code: loc ? new URL(loc).searchParams.get('code') : null, v };
}

async function token(params, ip = '10.0.0.5') {
  const r = await fetch(`${AUTH}/token`, { method: 'POST', headers: { Authorization: basic(WEB),
'X-Forwarded-For': ip }, body: new URLSearchParams(params) });
  return { status: r.status, corpo: await r.json() };
}

(async () => {
  await require('../src/auth-server').iniciar();
  await require('../src/api-financeira').iniciar();
  await require('../src/cliente-web').iniciar();
  console.log(`\nTestes do Laboratório 2 — auditoria e logs (pasta ${PASTA})\n`);

  console.log('1. Log de acesso e correlação');
  const r0 = await fetch(`${AUTH}/.well-known/oauth-authorization-server`, { headers: { 'X-Request-
```

```
Id': 'teste-correlacao-001' } });
  verificar('X-Request-Id válido é devolvido na resposta', r0.headers.get('x-request-id') ===
'teste-correlacao-001');
  verificar('HTTP_REQUISICAO registrada com o mesmo req_id', eventos('HTTP_REQUISICAO').some((e) =>
e.req_id === 'teste-correlacao-001' && e.status === 200));
  const r1 = await fetch(`${AUTH}/`, { headers: { 'X-Request-Id': 'x"}{inj' } });
  verificar('X-Request-Id inválido é substituído (anti-injeção)', /^[0-9a-f-]
{36}$/.test(r1.headers.get('x-request-id')));

  console.log('2. Eventos de autenticação');
  const falha = await login('10.1.1.1', 'senha-errada-xyz');
  const ev = eventos('LOGIN_FALHA').at(-1);
  verificar('login inválido -> LOGIN_FALHA (AVISO) com IP', falha.status === 401 && ev?.nivel ===
'AVISO' && ev?.ip === '10.1.1.1');
  verificar('login digitado é gravado cifrado e pode ser decifrado com a chave',
ev?.login_cifrado?.startsWith('gcm:') && auditoria.decifrar(ev.login_cifrado) === 'aluno');
  const l = await login('10.1.1.2');
  verificar('login válido -> LOGIN_SUCESSO', l.code && eventos('LOGIN_SUCESSO').at(-1)?.ip ===
'10.1.1.2');
  const t = await token({ grant_type: 'authorization_code', code: l.code, redirect_uri:
WEB.redirect_uris[0], code_verifier: l.v });
  const jti = jwt.decodificar(t.corpo.access_token).jti;
  verificar('TOKEN_EMITIDO traz o jti do token', eventos('TOKEN_EMITIDO').some((e) => e.jti ===
jti));

  console.log('3. Eventos da API');
  await fetch(`${API}/transacoes`, { headers: { Authorization: `Bearer ${t.corpo.access_token}` }
});
  verificar('ACESSO_PERMITIDO correlacionável ao token pelo jti',
eventos('ACESSO_PERMITIDO').some((e) => e.jti === jti && e.escopo === 'transacoes:read'));
  await fetch(`${API}/transferencias`, { method: 'POST', headers: { Authorization: `Bearer
${t.corpo.access_token}` } });
  verificar('403 -> ACESSO_NEGADO com escopo exigido', eventos('ACESSO_NEGADO').some((e) =>
e.status === 403 && e.escopo_exigido === 'transacoes:write'));
  const [h, , s] = t.corpo.access_token.split('.');
  const falso = Buffer.from(JSON.stringify({ ...jwt.decodificar(t.corpo.access_token), scope:
'transacoes:write' })).toString('base64url');
  await fetch(`${API}/transferencias`, { method: 'POST', headers: { Authorization: `Bearer
${h}.${falso}.${s}`, 'X-Forwarded-For': '198.51.100.23' } });
  verificar('token adulterado -> ALERTA_TOKEN_ADULTERADO (CRITICO)',
eventos('ALERTA_TOKEN_ADULTERADO').some((e) => e.nivel === 'CRITICO' && e.ip === '198.51.100.23'));
  verificar('alerta crítico também vai para alertas.log', fs.readFileSync(path.join(PASTA,
'alertas.log'), 'utf8').includes('ALERTA_TOKEN_ADULTERADO'));
```

```
  console.log('4. Força bruta: alerta e bloqueio');
  const st = [];
  for (let i = 0; i < 5; i++) st.push((await login('203.0.113.9', `chute${i}`)).status);
  verificar('5 falhas do mesmo IP -> ALERTA_FORCA_BRUTA', eventos('ALERTA_FORCA_BRUTA').some((e) =>
e.ip === '203.0.113.9'), st.join(','));
  verificar('IP bloqueado recebe 429 mesmo com a senha certa', (await login('203.0.113.9')).status
=== 429);
  verificar('bloqueio registrado como REQUISICAO_BLOQUEADA',
eventos('REQUISICAO_BLOQUEADA').some((e) => e.ip === '203.0.113.9'));
  verificar('outro IP continua conseguindo entrar', (await login('203.0.113.10')).status === 302);

  console.log('5. Refresh token reutilizado');
  const t2 = await token({ grant_type: 'refresh_token', refresh_token: t.corpo.refresh_token });
  await token({ grant_type: 'refresh_token', refresh_token: t.corpo.refresh_token },
'198.51.100.50');
  verificar('reuso -> REFRESH_REUSO (CRITICO) com impressão do token, não o token',
eventos('REFRESH_REUSO').some((e) => e.nivel === 'CRITICO' && /^sha256:[0-9a-f]
{12}$/.test(e.refresh_token)) && t2.status === 200);

  console.log('6. Nada sensível em claro nos arquivos');
  const tudo = auditoria.listarArquivos(PASTA).map((f) => fs.readFileSync(f, 'utf8')).join('') +
fs.readFileSync(path.join(PASTA, 'alertas.log'), 'utf8');
  const proibidos = { 'senha correta': 'senha123', 'senha errada': 'senha-errada-xyz',
client_secret: WEB.client_secret, access_token: t.corpo.access_token, refresh_token:
t.corpo.refresh_token, 'código de autorização': l.code };
  const vazou = Object.entries(proibidos).filter(([, v]) => tudo.includes(v)).map(([k]) => k);
  verificar('nenhuma senha, segredo, token ou code nos logs', vazou.length === 0, vazou.join(',
'));
  const linhas = tudo.split('\n').filter(Boolean);
  verificar('todas as linhas são JSON com ts, nivel, tipo e hash', linhas.every((x) => { try {
const o = JSON.parse(x); return o.ts && o.nivel && o.tipo && o.hash; } catch { return false; } }));

  console.log('7. Integridade (HMAC encadeado)');
  const integ = auditoria.verificarIntegridade(PASTA);
  verificar(`cadeia íntegra (${integ.total} registros)`, integ.ok,
JSON.stringify(integ.erros.slice(0, 2)));
  const copia = fs.mkdtempSync(path.join(os.tmpdir(), 'auditoria-adulterada-'));
  for (const f of auditoria.listarArquivos(PASTA)) fs.copyFileSync(f, path.join(copia,
path.basename(f)));
  const alvo = path.join(copia, 'auditoria.log');
  const ls = fs.readFileSync(alvo, 'utf8').split('\n');
  const iAlvo = ls.findIndex((x) => x.includes('"status":401'));
  ls[iAlvo] = ls[iAlvo].replace('"status":401', '"status":200');
  fs.writeFileSync(alvo, ls.join('\n'));
```

```
  const v1 = auditoria.verificarIntegridade(copia);
  verificar('edição de uma linha é detectada (linha exata)', !v1.ok && v1.erros.some((e) => e.linha
=== iAlvo + 1 && e.motivo.includes('alterado')));
  ls.splice(iAlvo, 2);
  fs.writeFileSync(alvo, ls.join('\n'));
  verificar('remoção de linhas é detectada (encadeamento)',
auditoria.verificarIntegridade(copia).erros.some((e) => e.motivo.includes('encadeamento')));

  console.log('8. Rotação e retenção');
  for (let i = 0; i < 40; i++) await fetch(`${API}/saldo`);
  const arquivos = auditoria.listarArquivos(PASTA);
  verificar('arquivo rotacionado ao passar de LOG_MAX_BYTES', arquivos.length >= 2,
arquivos.map((f) => path.basename(f)).join(','));
  verificar('rotação fica registrada no próximo evento', auditoria.lerEventos(PASTA).some((e) =>
e.rotacao?.arquivo_rotacionado));
  verificar('cadeia continua íntegra entre arquivos', auditoria.verificarIntegridade(PASTA).ok);

  console.log('9. Análise');
  const a = analisar(PASTA);
  verificar('IP de força bruta aparece como bloqueado na análise', a.suspeitos_por_ip.some((x) =>
x.ip === '203.0.113.9' && x.bloqueado));
  verificar('análise lista alertas críticos (>= 3)', a.alertas.length >= 3,
String(a.alertas.length));
  verificar('KPIs calculados (6 indicadores)', a.kpis.length === 6);

  console.log(`\nResultado: ${ok} OK, ${falhas} FALHA(S)\n`);
  process.exit(falhas ? 1 : 0);
})().catch((e) => {
  console.error(`Erro inesperado nos testes: ${e.stack || e.message}`);
  if (e.code === 'EADDRINUSE') console.error('Portas de teste 23000/24000/25000 ocupadas.');
  process.exit(2);
});
```

`testes/teste-fluxo.js` (124 linhas)

testes/teste-fluxo.js

```
// ============================================================
// teste-fluxo.js — Testes automáticos do laboratório (npm test)
// Sobe os servidores em portas de teste e verifica cada regra de
// segurança do OAuth. Saída: [OK] / [FALHA] e código de saída.
// ============================================================
process.env.AUTH_PORT = '14000';
process.env.API_PORT = '15000';
process.env.CLIENT_PORT = '13000';
```

```
process.env.OAUTH_QUIET = '1';
// Lab 2: logs dos testes vão para uma pasta temporária (não misturam com logs/ do aluno)
process.env.LOG_DIR =
require('node:fs').mkdtempSync(require('node:path').join(require('node:os').tmpdir(), 'oauth-teste-
'));

const cfg = require('../src/config');
const jwt = require('../src/jwt');
const { sha256b64url, aleatorio } = require('../src/util');

const AUTH = cfg.EMISSOR;
const API = `http://localhost:${cfg.PORTAS.api}`;
const WEB = cfg.CLIENTES[0];
const SVC = cfg.CLIENTES[1];
const REDIRECT = WEB.redirect_uris[0];
const basic = (c) => `Basic
${Buffer.from(`${c.client_id}:${c.client_secret}`).toString('base64')}`;

let ok = 0;
let falhas = 0;
function verificar(descricao, condicao, detalhe = '') {
  if (condicao) { ok++; console.log(`  [OK]    ${descricao}`); }
  else { falhas++; console.log(`  [FALHA] ${descricao}${detalhe ? ` -> ${detalhe}` : ''}`); }
}

async function token(params, cliente = WEB) {
  const r = await fetch(`${AUTH}/token`, { method: 'POST', headers: { Authorization: basic(cliente)
}, body: new URLSearchParams(params) });
  return { status: r.status, corpo: await r.json() };
}

async function obterCodigo(verifier, extra = {}) {
  const corpo = new URLSearchParams({
    response_type: 'code', client_id: WEB.client_id, redirect_uri: REDIRECT, scope: 'saldo:read
transacoes:read',
    state: 'estado123', code_challenge: sha256b64url(verifier), code_challenge_method: 'S256',
    login: 'aluno', senha: 'senha123', decisao: 'permitir', ...extra,
  });
  const r = await fetch(`${AUTH}/authorize`, { method: 'POST', body: corpo, redirect: 'manual' });
  const loc = r.headers.get('location');
  return loc ? new URL(loc).searchParams : null;
}

const getApi = (caminho, tk) => fetch(`${API}${caminho}`, { headers: tk ? { Authorization: `Bearer
```

```
${tk}` } : {} });

(async () => {
  await require('../src/auth-server').iniciar();
  await require('../src/api-financeira').iniciar();
  await require('../src/cliente-web').iniciar();
  console.log('\nTestes do laboratório OAuth\n');

  console.log('1. Registro e descoberta');
  const meta = await (await fetch(`${AUTH}/.well-known/oauth-authorization-server`)).json();
  verificar('metadados publicam token_endpoint e PKCE S256', meta.token_endpoint &&
meta.code_challenge_methods_supported.includes('S256'));
  verificar('App Finanças responde na porta do cliente', (await
fetch(`http://localhost:${cfg.PORTAS.cliente}/`)).status === 200);

  console.log('2. Pedido de autorização');
  const ruim = await fetch(`${AUTH}/authorize?${new URLSearchParams({ response_type: 'code',
client_id: WEB.client_id, redirect_uri: 'http://site-malicioso.exemplo/cb', scope: 'saldo:read',
state: 'x', code_challenge: 'abc', code_challenge_method: 'S256' })}`, { redirect: 'manual' });
  verificar('redirect_uri não registrada é recusada SEM redirecionar (400)', ruim.status === 400 &&
!ruim.headers.get('location'));
  const semPkce = await fetch(`${AUTH}/authorize?${new URLSearchParams({ response_type: 'code',
client_id: WEB.client_id, redirect_uri: REDIRECT, scope: 'saldo:read', state: 'x' })}`, { redirect:
'manual' });
  verificar('pedido sem PKCE volta com error=invalid_request', new
URL(semPkce.headers.get('location') || 'http://x').searchParams.get('error') ===
'invalid_request');
  const implicito = await fetch(`${AUTH}/authorize?${new URLSearchParams({ response_type: 'token',
client_id: WEB.client_id, redirect_uri: REDIRECT, scope: 'saldo:read', state: 'x' })}`, { redirect:
'manual' });
  verificar('Implicit Flow (response_type=token) é recusado', new
URL(implicito.headers.get('location') || 'http://x').searchParams.get('error') ===
'unsupported_response_type');
  const escopoIndevido = await obterCodigo('v'.repeat(43), { scope: 'relatorios:read' });
  verificar('escopo não permitido ao cliente -> invalid_scope', escopoIndevido?.get('error') ===
'invalid_scope');
  const negado = await obterCodigo('v'.repeat(43), { decisao: 'negar' });
  verificar('usuário que nega o consentimento -> access_denied', negado?.get('error') ===
'access_denied');

  console.log('3. Troca do código (Authorization Code + PKCE)');
  const v1 = aleatorio(32);
  const q1 = await obterCodigo(v1);
  verificar('login correto devolve code e o mesmo state', q1?.get('code') && q1.get('state') ===
```

```
'estado123');
  const pkceErrado = await token({ grant_type: 'authorization_code', code: q1.get('code'),
redirect_uri: REDIRECT, code_verifier: 'verificador-errado' });
  verificar('code_verifier errado -> invalid_grant', pkceErrado.status === 400 &&
pkceErrado.corpo.error === 'invalid_grant');
  const v2 = aleatorio(32);
  const q2 = await obterCodigo(v2);
  const segredoErrado = await token({ grant_type: 'authorization_code', code: q2.get('code'),
redirect_uri: REDIRECT, code_verifier: v2 }, { ...WEB, client_secret: 'errado' });
  verificar('client_secret errado -> 401 invalid_client', segredoErrado.status === 401);
  const v3 = aleatorio(32);
  const q3 = await obterCodigo(v3);
  const t1 = await token({ grant_type: 'authorization_code', code: q3.get('code'), redirect_uri:
REDIRECT, code_verifier: v3 });
  verificar('troca correta devolve access_token + refresh_token', t1.status === 200 &&
t1.corpo.access_token && t1.corpo.refresh_token);
  verificar('token traz apenas os escopos pedidos (menor privilégio)', t1.corpo.scope ===
'saldo:read transacoes:read');
  const reuso = await token({ grant_type: 'authorization_code', code: q3.get('code'), redirect_uri:
REDIRECT, code_verifier: v3 });
  verificar('código é de uso único (2ª troca -> invalid_grant)', reuso.status === 400);

  console.log('4. Servidor de recursos (API)');
  verificar('GET /transacoes com token válido -> 200', (await getApi('/transacoes',
t1.corpo.access_token)).status === 200);
  verificar('GET /transacoes sem token -> 401', (await getApi('/transacoes')).status === 401);
  const transf = await fetch(`${API}/transferencias`, { method: 'POST', headers: { Authorization:
`Bearer ${t1.corpo.access_token}` } });
  verificar('POST /transferencias sem escopo transacoes:write -> 403', transf.status === 403);
  const [h, , s] = t1.corpo.access_token.split('.');
  const pAdulterado = Buffer.from(JSON.stringify({ ...jwt.decodificar(t1.corpo.access_token),
scope: 'transacoes:write' })).toString('base64url');
  verificar('token adulterado (escopo alterado) -> 401', (await getApi('/transacoes',
`${h}.${pAdulterado}.${s}`)).status === 401);
  const expirado = jwt.assinar({ ...jwt.decodificar(t1.corpo.access_token), exp:
Math.floor(Date.now() / 1000) - 5 }, cfg.SEGREDO_ASSINATURA);
  verificar('token expirado -> 401', (await getApi('/transacoes', expirado)).status === 401);

  console.log('5. Renovação (refresh token com rotação)');
  const t2 = await token({ grant_type: 'refresh_token', refresh_token: t1.corpo.refresh_token });
  verificar('refresh válido devolve NOVO par de tokens', t2.status === 200 &&
t2.corpo.refresh_token !== t1.corpo.refresh_token);
  const reusoRt = await token({ grant_type: 'refresh_token', refresh_token: t1.corpo.refresh_token
});
```

```
  verificar('reuso do refresh antigo é detectado (invalid_grant)', reusoRt.status === 400 &&
reusoRt.corpo.error_description.includes('REUSO'));
  const t3 = await token({ grant_type: 'refresh_token', refresh_token: t2.corpo.refresh_token });
  verificar('após reuso, o refresh novo também foi revogado (família)', t3.status === 400);

  console.log('6. Client Credentials (servidor-servidor)');
  const cc = await token({ grant_type: 'client_credentials', scope: 'relatorios:read' }, SVC);
  verificar('serviço obtém token sem usuário e sem refresh_token', cc.status === 200 &&
!cc.corpo.refresh_token);
  verificar('GET /relatorios/resumo com token do serviço -> 200', (await
getApi('/relatorios/resumo', cc.corpo.access_token)).status === 200);
  verificar('token do serviço não acessa /transacoes -> 403', (await getApi('/transacoes',
cc.corpo.access_token)).status === 403);
  const ccWeb = await token({ grant_type: 'client_credentials' }, WEB);
  verificar('App Finanças não pode usar client_credentials -> unauthorized_client',
ccWeb.corpo.error === 'unauthorized_client');

  console.log(`\nResultado: ${ok} OK, ${falhas} FALHA(S)\n`);
  process.exit(falhas ? 1 : 0);
})().catch((e) => {
  console.error(`Erro inesperado nos testes: ${e.stack || e.message}`);
  if (e.code === 'EADDRINUSE') console.error('Portas de teste 13000/14000/15000 ocupadas.');
  process.exit(2);
});
```

Resultado esperado: Duas linhas: `25 OK, 0 FALHA(S)` e `27 OK, 0 FALHA(S)`.

✔ EXEMPLO DE SUCESSO

```
Resultado: 25 OK, 0 FALHA(S)
Resultado: 27 OK, 0 FALHA(S)
```

✘ EXEMPLO DE FALHA TÍPICA

```
  [FALHA] nenhuma senha, segredo, token ou code nos logs -> senha errada
```

Como corrigir: Algum evento grava dado sensível em claro: confira o nome do campo (mascarar() age pelo nome) ou use cifrar().

Passo 10 Registrar a instrumentação no Git

Git

Crie também a configuração de referência do Filebeat e o modelo do plano antes do commit (serão usados nos passos 17 e 18).

Git — comandos

```
git add .
git commit -m "feat: auditoria e logs da API (eventos, alertas, integridade, rotacao)"
git log --oneline -3
```

`config/filebeat.yml` (40 linhas)

config/filebeat.yml

```
# ============================================================
# config/filebeat.yml — EXEMPLO (opcional, não executado no lab)
# Envia logs/auditoria.log (JSON Lines) para o Elasticsearch,
# onde o Kibana permite filtrar, correlacionar e criar alertas.
# Pré-requisitos (fora do laboratório): Elastic Stack instalado
# ou conta no Elastic Cloud, e Filebeat 8.x.
# Uso: filebeat -e -c config\filebeat.yml
# ============================================================
filebeat.inputs:
  - type: filestream
    id: api-financeira-auditoria
    paths:
      - ${PASTA_PROJETO:.}/logs/auditoria*.log   # atual + rotacionados
    parsers:
      - ndjson:
          target: ""            # campos do JSON vão para a raiz do documento
          add_error_key: true
    fields:
      servico: api-financeira
      ambiente: laboratorio
    fields_under_root: true

processors:
  - timestamp:                 # usa o "ts" do evento como @timestamp
      field: ts
      layouts: ['2006-01-02T15:04:05.999Z']
  - drop_fields:
      fields: ["hash_anterior"]  # o hash fica; o elo anterior não é útil na busca

output.elasticsearch:
  hosts: ["${ELASTIC_HOST:http://localhost:9200}"]
  username: "${ELASTIC_USUARIO}"
  password: "${ELASTIC_SENHA}"   # NUNCA grave a senha aqui: use variável de ambiente
  index: "auditoria-api-%{+yyyy.MM.dd}"

setup.template.name: "auditoria-api"
```

```
setup.template.pattern: "auditoria-api-*"
# Retenção no Elasticsearch: crie uma política ILM (ex.: apagar após 90 dias)
setup.ilm.enabled: false
```

`docs/plano-auditoria.md` (58 linhas)

docs/plano-auditoria.md

```
# Plano de auditoria e logs — API Financeira

**Grupo:** _(nomes dos integrantes)_  **Turma:** _____  **Data:** ___/___/_____

## 1. Contexto e objetivo
A API financeira (provedor OAuth, API de recursos e App Finanças) processa logins, emissão de tokens, consultas e transferências. O objetivo do plano é **registrar o que importa, proteger os registros e analisá-los periodicamente** para detectar e responder a incidentes.

## 2. Ferramentas de auditoria (seleção e justificativa)
| Camada | No laboratório | Em produção (recomendado) | Por quê |
|---|---|---|---|
| Geração | `src/auditoria.js` (JSON Lines) | Biblioteca de log estruturado (ex.: pino) com o mesmo formato | Um evento por linha, campos fixos: qualquer ferramenta lê sem conversão |
| Coleta/centralização | Pasta `logs/` | Filebeat → **ELK Stack** (Elasticsearch + Kibana) ou **Splunk** | Consolida logs de vários servidores; busca e correlação em segundos |
| Análise | `npm run analisar` (KPIs, correlação, relatório) | Dashboards do Kibana/Splunk + consultas salvas | Visualização, filtros e indicadores |
| Alerta | Console + `logs/alertas.log` + webhook | Regras do SIEM → Slack/Teams/e-mail/plantão | Resposta rápida a eventos críticos |

## 3. Eventos registrados
| Categoria | Eventos | Nível |
|---|---|---|
| Acesso (todas as requisições) | `HTTP_REQUISICAO` (método, rota, status, duração, IP, req_id) | INFO/AVISO/ERRO |
| Autenticação | `LOGIN_SUCESSO`, `LOGIN_FALHA`, `CLIENTE_AUTH_FALHA`, `CONSENTIMENTO_NEGADO` | INFO/AVISO |
| Tokens | `TOKEN_EMITIDO` (com `jti`), `TOKEN_RECUSADO`, `TOKEN_REVOGADO`, `REFRESH_REUSO` | INFO/AVISO/**CRITICO** |
| Autorização | `ACESSO_PERMITIDO`, `ACESSO_NEGADO` (401/403 + motivo), `AUTORIZACAO_RECUSADA` | INFO/AVISO |
| Operações sensíveis | `OPERACAO_SENSIVEL` (transferências) | INFO |
| Ataques detectados | `ALERTA_FORCA_BRUTA`, `ALERTA_TOKEN_ADULTERADO`, `REQUISICAO_BLOQUEADA` | **CRITICO**/AVISO |
| Falhas | `ERRO_SERVIDOR` | ERRO |
```

```
**Nunca registrar em claro:** senhas, `client_secret`, tokens, código de autorização, query string. Tokens viram impressão digital (`sha256:…`); o login digitado errado vai **cifrado (AES-256-GCM)**.

## 4. Configuração e proteção dos logs
- **Integridade:** HMAC encadeado — cada linha assina o conteúdo + o hash da anterior; edição ou remoção é apontada pela verificação.
- **Confidencialidade:** arquivos com permissão restrita, dados sensíveis cifrados, chaves em cofre/variável de ambiente, `logs/` fora do Git.
- **Rotação:** novo arquivo ao atingir `LOG_MAX_BYTES` (lab: 256 KB; produção: diária ou 100 MB).
- **Retenção:** manter `LOG_RETENCAO_ARQUIVOS` arquivos / `LOG_RETENCAO_DIAS` dias localmente; no SIEM, 90 dias quentes + 1 ano arquivado (ajustar à LGPD e às normas do setor financeiro).
- **Correlação:** `X-Request-Id` propagado do App Finanças ao provedor e à API; `jti` liga o token emitido a cada uso.

## 5. Análise periódica e resposta
| Frequência | O que fazer | Indicador / meta |
|---|---|---|
| Tempo real | Alertas críticos (força bruta, token adulterado, reuso de refresh) | 0 alertas não tratados |
| Diária | `npm run analisar`: KPIs, IPs suspeitos, recomendações | Falha de login < 20%; erro HTTP < 10% |
| Semanal | Revisar 403 (escalonamento), novos IPs, clientes com falhas | Tendência estável |
| Mensal | Verificar integridade de todos os arquivos, revisar regras e limites | Integridade 100% |

**Procedimento de incidente:** (1) alerta recebido → (2) rastrear por IP/`req_id`/`jti` → (3) conter (bloquear IP, revogar tokens) → (4) comunicar → (5) registrar lições aprendidas e ajustar regras.

## 6. Evidências do laboratório
- [ ] `npm test` com 25 + 27 OK
- [ ] Saída do `npm run simular` com os 429
- [ ] Relatório `relatorio/relatorio-auditoria.html` (print dos KPIs e alertas)
- [ ] Bloqueio pelo navegador (5 senhas erradas → 429)
- [ ] Adulteração detectada pelo analisador (linha apontada)
- [ ] Rotação observada com `LOG_MAX_BYTES` pequeno

## Referências
- SEDUC-SP. *Rastreando ameaças: auditoria e logs para APIs seguras* — Programação Back-End, 2ª série, 4º bimestre, semana 25, aula 3.
- [OWASP Logging Cheat Sheet] (https://cheatsheetseries.owasp.org/cheatsheets/Logging_Cheat_Sheet.html)
```

```
- [NIST SP 800-92 — Guide to Computer Security Log Management]
(https://csrc.nist.gov/pubs/sp/800/92/final)
```

Resultado esperado: Commit na branch lab2-auditoria; `git status` não lista a pasta logs/.

✔ EXEMPLO DE SUCESSO

```
[lab2-auditoria 8c41d2e] feat: auditoria e logs da API ...
```

✘ EXEMPLO DE FALHA TÍPICA

```
logs/auditoria.log aparece em git status
```

Como corrigir: Faltou a linha logs/ no .gitignore; depois rode git rm -r --cached logs.

Passo 11 Iniciar e observar os logs ao vivo

PowerShell (2 terminais) + Navegador

No 1º terminal suba os servidores. No 2º acompanhe o log em tempo real. Use o App Finanças (login, consultas). Na aba Rede do DevTools, copie o cabeçalho `X-Request-Id` de uma resposta e rastreie-o nos três serviços.

PowerShell (2 terminais) + Navegador — comandos

```
# terminal 1
npm start
# terminal 2
Get-Content logs\auditoria.log -Wait -Tail 5
# rastrear uma requisição (Ctrl+C no Get-Content antes)
node src\analisar-logs.js --req=<cole-o-X-Request-Id>
```

Resultado esperado: Eventos HTTP_REQUISICAO, LOGIN_SUCESSO, TOKEN_EMITIDO e ACESSO_PERMITIDO aparecendo; o rastreio lista eventos de cliente, auth e api.

✔ EXEMPLO DE SUCESSO

```
{"ts":"2026-11-
03T13:02:11.481Z","nivel":"INFO","tipo":"LOGIN_SUCESSO","origem":"auth","sub":"u1
",...,"req_id":"5f0c..."}
```

✘ EXEMPLO DE FALHA TÍPICA

```
Get-Content : Não é possível localizar o caminho '...\logs\auditoria.log'
```

Como corrigir: O arquivo só nasce na 1ª requisição: abra http://localhost:3000 e repita.

Passo 12 Provocar força bruta pelo navegador

Navegador + PowerShell

Clique em Sair e tente entrar **5 vezes com senha errada**. Na 5ª, o terminal do servidor mostra o alerta em vermelho; na 6ª, mesmo com a senha certa, o provedor responde **429**. Para liberar antes dos 5 minutos, reinicie o servidor (os bloqueios ficam em memória — discuta por que em produção isso precisa de um armazenamento compartilhado, como Redis).

Navegador + PowerShell — comandos

```
Get-Content logs\alertas.log -Tail 3
```

Resultado esperado: Linha ALERTA_FORCA_BRUTA com o seu IP (::1 ou 127.0.0.1) e tela de bloqueio no navegador.

✔ EXEMPLO DE SUCESSO

```
[ALERTA CRITICO] ... ALERTA_FORCA_BRUTA ip=::1 IP bloqueado por 300s
429 Too Many Requests
```

✘ EXEMPLO DE FALHA TÍPICA

```
Nenhum alerta após 5 erros
```

Como corrigir: As falhas precisam ocorrer em menos de 60 s (JANELA_S) e no mesmo servidor em execução.

Passo 13 Simular tráfego normal e ataques

PowerShell (2º terminal)

Com o servidor rodando, gere tráfego de 3 usuários, do serviço de relatórios e de cinco ataques vindos de IPs diferentes.

PowerShell (2º terminal) — comandos

```
npm run simular
```

Resultado esperado: Força bruta termina em 429 e o terminal do servidor exibe os alertas críticos.

✔ EXEMPLO DE SUCESSO

```
status recebidos: 401, 401, 401, 401, 401, 429, 429, 429
Concluído: 18 requisições normais e 26 de ataque
```

✘ EXEMPLO DE FALHA TÍPICA

```
Erro: fetch failed. O laboratório está rodando?
```

Como corrigir: Deixe npm start rodando no terminal 1 e execute npm run simular no terminal 2.

Passo 14 Analisar os logs

PowerShell + Navegador

Gere os indicadores e o relatório. Use `--decifrar` para ver o que foi digitado nas falhas de login (acesso restrito) e responda: qual IP deve ser bloqueado no firewall? Qual cliente tentou acessar fora do escopo?

PowerShell + Navegador — comandos

```
npm run analisar
Start-Process relatorio\relatorio-auditoria.html
node src\analisar-logs.js --decifrar
```

Resultado esperado: KPIs com ATENÇÃO, alertas listados e 203.0.113.66 / 203.0.113.99 marcados como bloqueados.

✔ EXEMPLO DE SUCESSO

```
[ATENCAO] Alertas críticos: 6 (meta 0)
  203.0.113.66   9 [BLOQUEADO] LOGIN_FALHA, ...
```

✘ EXEMPLO DE FALHA TÍPICA

```
Nenhum log em ...\logs. Rode "npm start" e gere tráfego
```

Como corrigir: Execute o passo 13 antes (ou rode a análise na pasta do projeto).

Passo 15 Testar a integridade dos logs

PowerShell + VS Code

Faça uma cópia de segurança, abra `logs\auditoria.log` no VS Code e troque um `"status":401` por `"status":200` (como faria um invasor apagando rastros). Salve e rode a análise: a linha exata é apontada e o código de saída vira 2. Depois restaure a cópia.

PowerShell + VS Code — comandos

```
Copy-Item logs\auditoria.log logs\copia-auditoria.txt
code logs\auditoria.log        # edite e salve
npm run analisar; $LASTEXITCODE   # esperado: 2
```

```
Copy-Item logs\copia-auditoria.txt logs\auditoria.log -Force
npm run analisar; $LASTEXITCODE   # esperado: 0
```

Resultado esperado: “Integridade: VIOLADA … linha N: conteúdo alterado” e, após restaurar, OK.

✔ EXEMPLO DE SUCESSO

```
Integridade: VIOLADA (...)
   ! auditoria.log linha 37: conteúdo alterado (HMAC não confere)
2
```

✘ EXEMPLO DE FALHA TÍPICA

```
Integridade: OK  (a edição não foi salva)
```

Como corrigir: Salve o arquivo no VS Code (Ctrl+S) antes de rodar a análise.

Passo 16 Observar rotação e retenção

PowerShell

Reinicie com arquivo máximo de 8 KB e retenção de 3 arquivos, gere tráfego duas vezes e liste a pasta. A rotação fica registrada no campo `rotacao` do evento seguinte; a análise continua íntegra e avisa que o início da cadeia foi expurgado.

PowerShell — comandos

```
# Ctrl+C no terminal do servidor, depois:
$env:LOG_MAX_BYTES = 8000; $env:LOG_RETENCAO_ARQUIVOS = 3
npm start
# no 2º terminal:
npm run simular; npm run simular
Get-ChildItem logs
npm run analisar
# ao terminar (terminal 1): Ctrl+C e
Remove-Item Env:LOG_MAX_BYTES, Env:LOG_RETENCAO_ARQUIVOS
```

Resultado esperado: No máximo 3 arquivos auditoria-AAAAMMDDT…-NNN.log além do auditoria.log.

✔ EXEMPLO DE SUCESSO

```
auditoria-20261103T131502117-000.log ... (3 arquivos) + auditoria.log
Integridade: OK
```

✘ EXEMPLO DE FALHA TÍPICA

`Só existe auditoria.log`

Como corrigir: As variáveis precisam ser definidas no MESMO terminal antes do npm start.

Passo 17 (Opcional) Centralizar e notificar

VS Code + serviços externos

Leia `config/filebeat.yml`: é como estes mesmos logs seriam enviados ao **ELK Stack** (ou, de forma análoga, ao Splunk). Se a turma tiver um canal de testes no Slack/Teams com webhook, configure `ALERTA_WEBHOOK` e repita o passo 12. **Etapa manual**: exige conta/instalação externas; o script não configura.

VS Code + serviços externos — comandos

```
$env:ALERTA_WEBHOOK = "https://hooks.slack.com/services/SEU/WEBHOOK/AQUI"
npm start
```

Resultado esperado: Mensagem de alerta chega no canal (ou o grupo descreve o fluxo no plano).

✔ EXEMPLO DE SUCESSO

`Mensagem “[ALERTA CRITICO] … ALERTA_FORCA_BRUTA …” no canal`

✘ EXEMPLO DE FALHA TÍPICA

`[auditoria] webhook falhou: fetch failed`

Como corrigir: Confira a URL do webhook e a rede da escola (proxy pode bloquear). É etapa opcional.

Passo 18 Redigir o plano de auditoria e logs

VS Code

Etapa **Execução** do roteiro: complete `docs/plano-auditoria.md` com os nomes do grupo, adapte as justificativas com as palavras do grupo e anexe as evidências dos passos 9 e 11 a 16. O plano precisa cobrir: seleção de ferramentas, eventos e configuração dos logs, e o procedimento de análise periódica.

Resultado esperado: As seis seções do plano preenchidas.

✔ EXEMPLO DE SUCESSO

`docs/plano-auditoria.md com as 6 seções preenchidas e evidências`

✘ EXEMPLO DE FALHA TÍPICA

```
Plano copiado do modelo sem adaptação e sem prints
```

Como corrigir: Reescreva com as palavras do grupo e anexe os prints dos passos 9 e 11–16.

Passo 19 Responder às perguntas e entregar

Git + VS Code

Responda às quatro perguntas da mensagem do Slack (o bloco de Respostas desta página serve de conferência após 50%), registre o plano e entregue. Se a turma usa repositório remoto, a publicação é manual: `git push -u origin lab2-auditoria` após login no GitHub.

Git + VS Code — comandos

```
git add docs
git commit -m "docs: plano de auditoria e logs do grupo"
git log --oneline -4
```

Resultado esperado: Commit do plano na branch lab2-auditoria e respostas entregues.

✔ EXEMPLO DE SUCESSO

```
a1b2c3d docs: plano de auditoria e logs do grupo
8c41d2e feat: auditoria e logs da API ...
```

✘ EXEMPLO DE FALHA TÍPICA

```
nothing to commit, working tree clean
```

Como corrigir: Salve o arquivo no VS Code (Ctrl+S) antes do git add.

Rubrica de avaliação (10 pontos)

Item	Critério de certo	Pontuação	Máx.
E1 — Instrumentação e testes (passos 3–10)	Print do `npm test` com 25 OK e 27 OK e commit na branch lab2-auditoria.	Certo: 1,0 · Errado: 0	1,0
E2 — Logs ao vivo e correlação (passo 11)	Print de eventos no `Get-Content -Wait` e de um rastreio `--req` com eventos de 2+ serviços.	0,5 por evidência	1,0

Item	Critério de certo	Pontuação	Máx.
E3 — Detecção e bloqueio (passos 12–13)	Print do alerta de força bruta/429 no navegador e da saída do `npm run simular`.	0,5 por evidência	**1,0**
E4 — Análise, integridade e rotação (14–16)	Print do relatório HTML, da violação de integridade detectada (linha) e da pasta logs com arquivos rotacionados.	Os 3 itens: 1,0 · 2 itens: 0,5 · ≤1: 0	**1,0**
E5 — Plano de auditoria (18–19)	`docs/plano-auditoria.md` com ferramentas justificadas, eventos e proteção dos logs, procedimento de análise periódica e evidências; commit.	0,5 por item presente	**2,0**
P1 — Ferramentas	Indica centralização (ELK/Splunk ou similar) + coletor ou log estruturado **e** justifica (consolidação, busca/correlação, alertas).	Certo: 1,0 · Sem justificativa: 0,5 · Errado: 0	**1,0**
P2 — Configuração e proteção	Cita eventos críticos com campos mínimos, o que NÃO registrar, integridade e retenção/rotação.	4 elementos: 1,0 · 2–3: 0,5 · ≤1: 0	**1,0**
P3 — Métodos de análise	Cita ao menos 3: limiar em janela, assinaturas, correlação, indicadores/linha de base, visualização.	≥3: 1,0 · 2: 0,5 · ≤1: 0	**1,0**
P4 — Comunicação entre equipes	Propõe ao menos 3 mecanismos concretos (alerta em canal, runbook/plantão, severidade comum, dashboard/revisão, pós-incidente).	≥3: 1,0 · 2: 0,5 · ≤1: 0	**1,0**
Total			**10,0**

Respostas da atividade

Caso prático (roteiro, item 2)

2.1 — Seleção de ferramentas de auditoria

Logs em **JSON Lines** gerados pela própria API (no lab, `src/auditoria.js`), coletados pelo **Filebeat** e centralizados no **ELK Stack** (Elasticsearch para busca e retenção, Kibana

para dashboards e alertas). O ELK é a escolha para a escola e para equipes menores por ser aberto e amplamente documentado; **Splunk** é alternativa comercial com correlação e alertas prontos. Regras de detecção em tempo real ficam na aplicação (bloqueio imediato) e no SIEM (visão consolidada de várias APIs). No laboratório, o analisador (`npm run analisar`) faz o papel do dashboard.

2.2 — Eventos registrados e configuração

- **Eventos:** todas as requisições (log de acesso), logins com sucesso e falha, falhas de autenticação de cliente, emissão/recusa/revogação de tokens, acessos permitidos e negados (401/403 com motivo), operações sensíveis (transferências), reuso de refresh token, tokens adulterados e erros 500.
- **Campos:** `ts` (UTC), `nivel`, `tipo`, `origem`, `ip`, `sub`, `client_id`, `rota`, `status`, `req_id`, `jti`, `motivo`.
- **Proteção:** nunca gravar senha, segredo, token ou query string; impressão digital para tokens; cifra AES-256-GCM para o login digitado; HMAC encadeado; permissão restrita; `logs/` fora do Git; chaves em cofre.
- **Armazenamento:** rotação por tamanho (ou diária) e retenção definida (ex.: 90 dias consultáveis + 1 ano arquivado, conforme LGPD e normas do setor).

2.3 — Procedimento de análise periódica

1. **Tempo real:** alertas críticos (força bruta, token adulterado, reuso de refresh) → plantão.
2. **Diário:** rodar a análise, conferir KPIs (falha de login < 20%, erro HTTP < 10%, 0 alertas sem tratamento, integridade 100%) e IPs suspeitos.
3. **Investigação:** rastrear por `ip`, `req_id` e `jti`; decifrar logins apenas quando necessário, com registro de quem acessou.
4. **Resposta:** conter (bloquear IP no firewall, revogar tokens, avisar usuário), registrar o incidente e ajustar regras.
5. **Mensal:** verificar a integridade de todos os arquivos, revisar limites e a política de retenção.

Exemplo do lab: a análise mostrou 203.0.113.66 com 5 falhas de login e 3 requisições bloqueadas em segundos — força bruta confirmada; recomendação: bloqueio permanente no firewall.

Perguntas da atividade

---

1. Quais ferramentas de auditoria vocês recomendariam para centralizar e

analisar os logs da nossa API, e por quê?

Recomendamos uma cadeia de três partes: **(1) log estruturado na própria API** (JSON Lines, um evento por linha com campos padronizados), **(2) um coletor** como o **Filebeat**, e **(3) uma plataforma de centralização e análise** — o **ELK Stack** (Elasticsearch + Kibana), ou o **Splunk** se a empresa preferir uma solução comercial com suporte. Motivos: consolidam os logs de várias APIs e servidores num só lugar (como no exemplo prático do material), permitem buscar e correlacionar eventos em segundos, criar dashboards com indicadores e configurar alertas para eventos críticos. O formato estruturado é o que torna tudo isso possível: no laboratório os mesmos arquivos são lidos pelo nosso analisador e seriam lidos pelo Filebeat sem nenhuma conversão (`config/filebeat.yml`). Para a resposta imediata, mantemos também regras na própria aplicação (bloqueio automático de força bruta).

---

2. Como vocês configurariam os logs para garantir que os eventos críticos sejam devidamente registrados e protegidos?

- **O que registrar:** tentativas de acesso (sucesso e falha), emissão e recusa de tokens, acessos negados com motivo, operações sensíveis, erros e sinais de ataque — sempre com data/hora UTC, IP, usuário, cliente, rota, status e um `req_id` para correlação.
- **O que nunca registrar:** senhas, segredos, tokens completos e query strings; usar impressão digital (hash) para tokens e **cifrar** dados sensíveis (no lab, AES-256-GCM no login digitado).
- **Integridade:** assinatura HMAC encadeada por linha (qualquer edição ou remoção é apontada — passo 15), envio imediato para o servidor central e acesso de escrita restrito.
- **Segurança do arquivo:** permissão mínima, pasta fora do controle de versão, chaves em cofre/variáveis de ambiente, validação do `X-Request-Id` contra injeção.
- **Rotação e retenção:** novo arquivo por tamanho ou por dia, retenção definida em política (ex.: 90 dias + arquivamento), respeitando a LGPD.

---

3. Quais métodos de análise de dados vocês utilizariam para identificar incidentes de segurança a partir dos logs?

- **Regras de limiar em janela de tempo:** N falhas do mesmo IP em X segundos → força bruta (no lab: 5 em 60 s → alerta e bloqueio 429).
- **Assinaturas de ataque:** eventos que por si só indicam incidente — token com assinatura inválida, reuso de refresh token.
- **Correlação:** agrupar por IP, usuário, cliente, `req_id` e `jti` para reconstruir o que aconteceu (no lab: `--req=<id>` mostra a mesma ação nos três serviços).
- **Indicadores e linha de base:** taxa de falha de login, taxa de erro HTTP, latência p95, alertas críticos e IPs bloqueados, comparados com metas; desvios da média

(picos de 403 ou 401) disparam investigação.
- **Visualização:** linha do tempo de eventos por minuto e ranking de IPs suspeitos.
- **Verificação de integridade** periódica e **revisões** para ajustar limites e regras.

---

4. Como vocês sugeririam melhorar a comunicação entre as equipes de segurança e TI para garantir uma resposta rápida a incidentes detectados pelos logs?

- **Alertas automáticos num canal compartilhado** (Slack/Teams via webhook — `ALERTA_WEBHOOK` no lab) com o essencial: tipo, IP, horário e `req_id` para rastreio.
- **Plano de resposta (runbook)** por tipo de alerta: quem é acionado, primeiros passos (bloquear IP, revogar tokens) e prazo, com plantão definido.
- **Classificação de severidade comum** (INFO, AVISO, ERRO, CRÍTICO) para as duas equipes falarem a mesma língua.
- **Dashboard único** com os mesmos indicadores para segurança e TI e reunião curta de revisão semanal.
- **Pós-incidente sem culpados:** registrar linha do tempo, causa e ajustes nas regras de log e alerta.

> "Equipe, proposta: centralizar os logs da API no ELK com Filebeat; alertas críticos (força bruta, token adulterado, reuso de refresh) vão para #seg-incidentes com IP e req_id; bloqueio automático após 5 falhas/60 s; análise diária com KPIs e verificação de integridade mensal. Plano e runbook em docs/plano-auditoria.md."

Fontes

- SEDUC-SP — Roteiro de Atividade Prática "Rastreando ameaças: auditoria e logs para APIs seguras" (SISANO2C2B4S25A3AP), Programação Back-End — material de referência
- OWASP Logging Cheat Sheet
  https://cheatsheetseries.owasp.org/cheatsheets/Logging_Cheat_Sheet.html
- OWASP Top 10 (2021) — A09 Security Logging and Monitoring Failures
  https://owasp.org/Top10/A09_2021-Security_Logging_and_Monitoring_Failures/
- NIST SP 800-92 — Guide to Computer Security Log Management
  https://csrc.nist.gov/pubs/sp/800/92/final
- Elastic — Filebeat: filestream input
  https://www.elastic.co/guide/en/beats/filebeat/current/filebeat-input-filestream.html
- Splunk — documentação
  https://docs.splunk.com/Documentation/Splunk
- JSON Lines — formato de texto com um JSON por linha

https://jsonlines.org/

- Lei nº 13.709/2018 — Lei Geral de Proteção de Dados (LGPD)
  https://www.planalto.gov.br/ccivil_03/_ato2015-2018/2018/lei/l13709.htm
- Node.js — módulo crypto (HMAC, AES-GCM)
  https://nodejs.org/api/crypto.html
- Laboratório — seção 1 (aula 1): Descomplicando o OAuth (atividade-oauth-api)

Feedback do aluno

O que funcionou, o que foi difícil e o que você mudaria nesta atividade?